



The Importance of Storage Performance in Inference Environments

May 22, 2026

Shuichi Ihara
Principal Engineer

[ANY DATA] [ANY APPLICATION] [ANYWHERE]

Tokens / \$

What Is Token Throughput?

Number of tokens produced per unit of time



$$\text{Token throughput} = \frac{\text{Input (output) token count}}{\text{Time taken for input (output)}}$$

How to Maximize Tokens / \$

If token count is fixed, what remains is “how fast tokens can be produced”

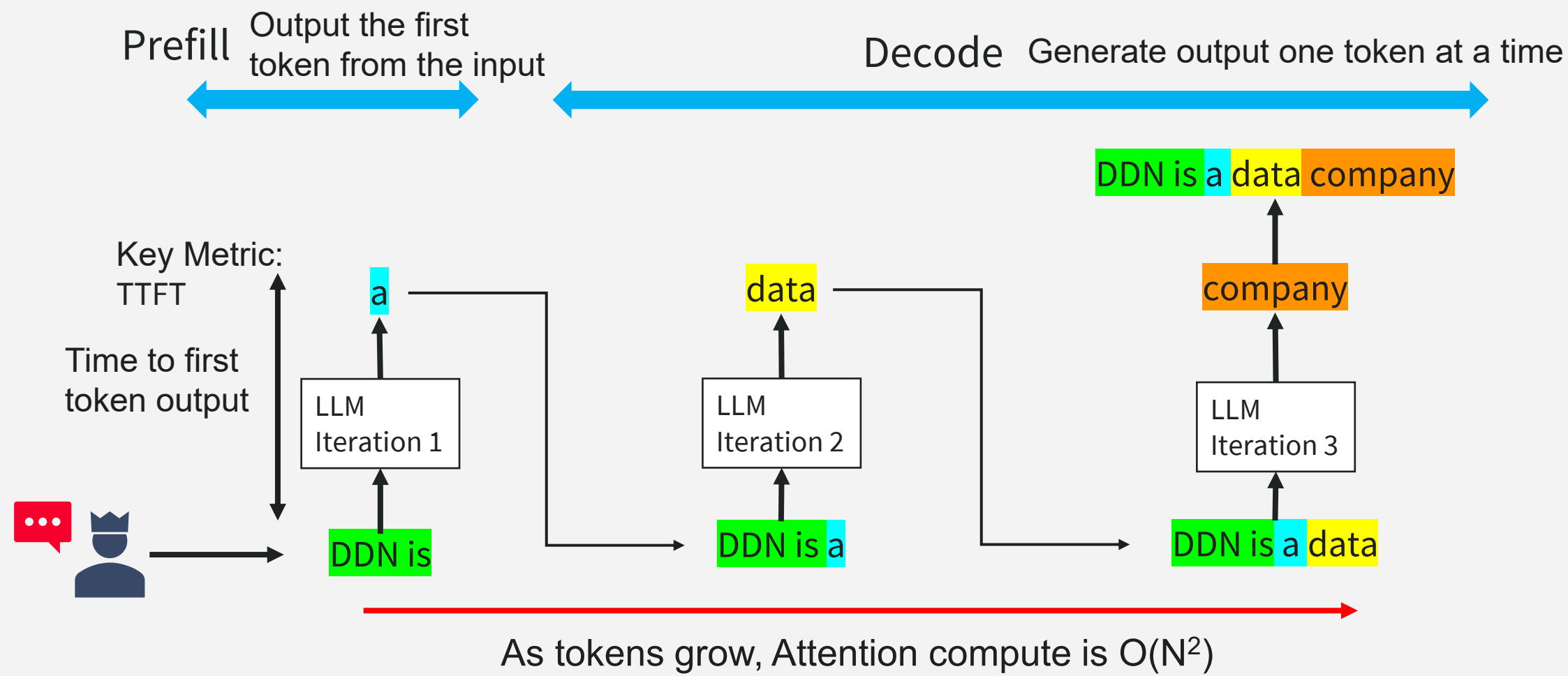


$$\text{Tokens / \$} = \frac{\text{Token throughput}}{\text{Hardware unit cost (\$/hour)}}$$

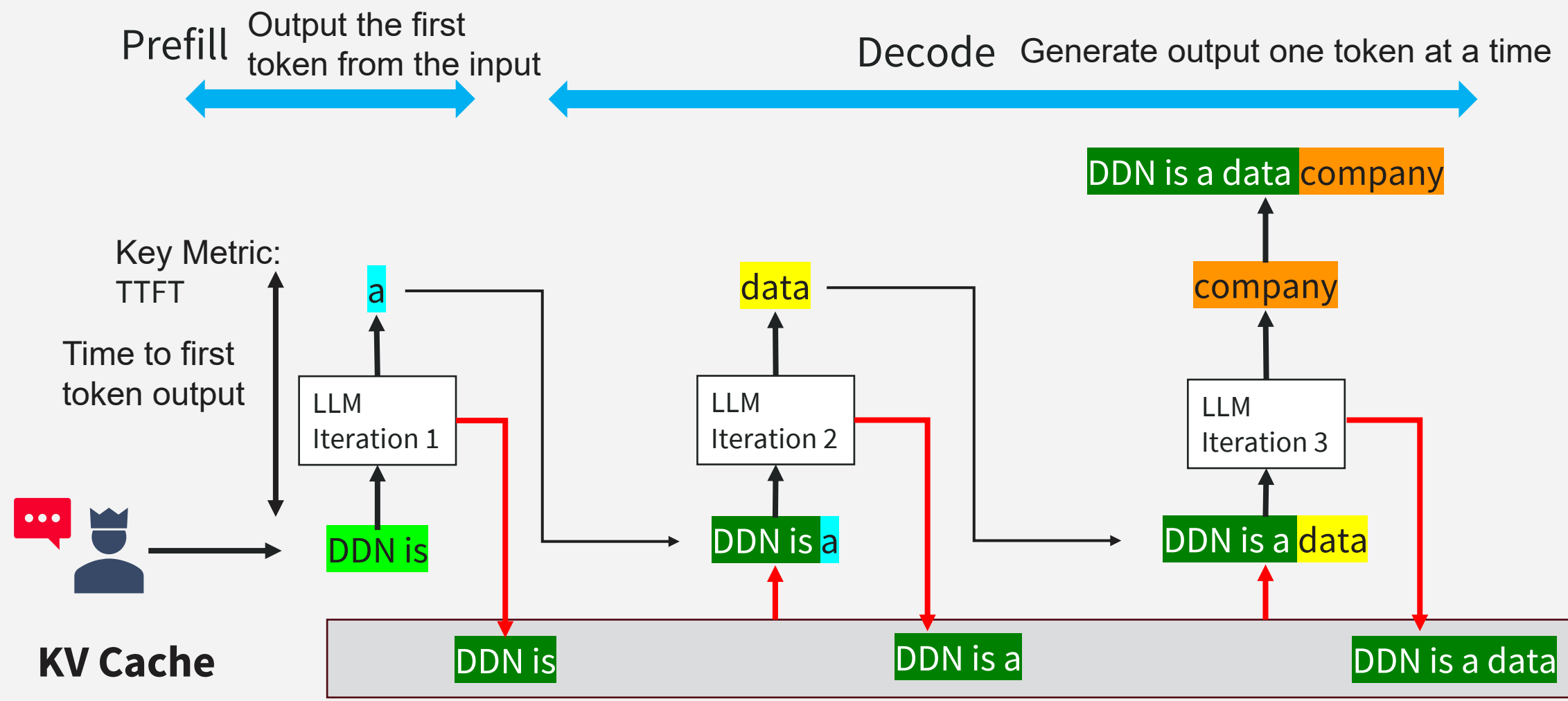
To maximize Tokens / \$:

- Maximize token throughput
- Minimize hardware unit cost

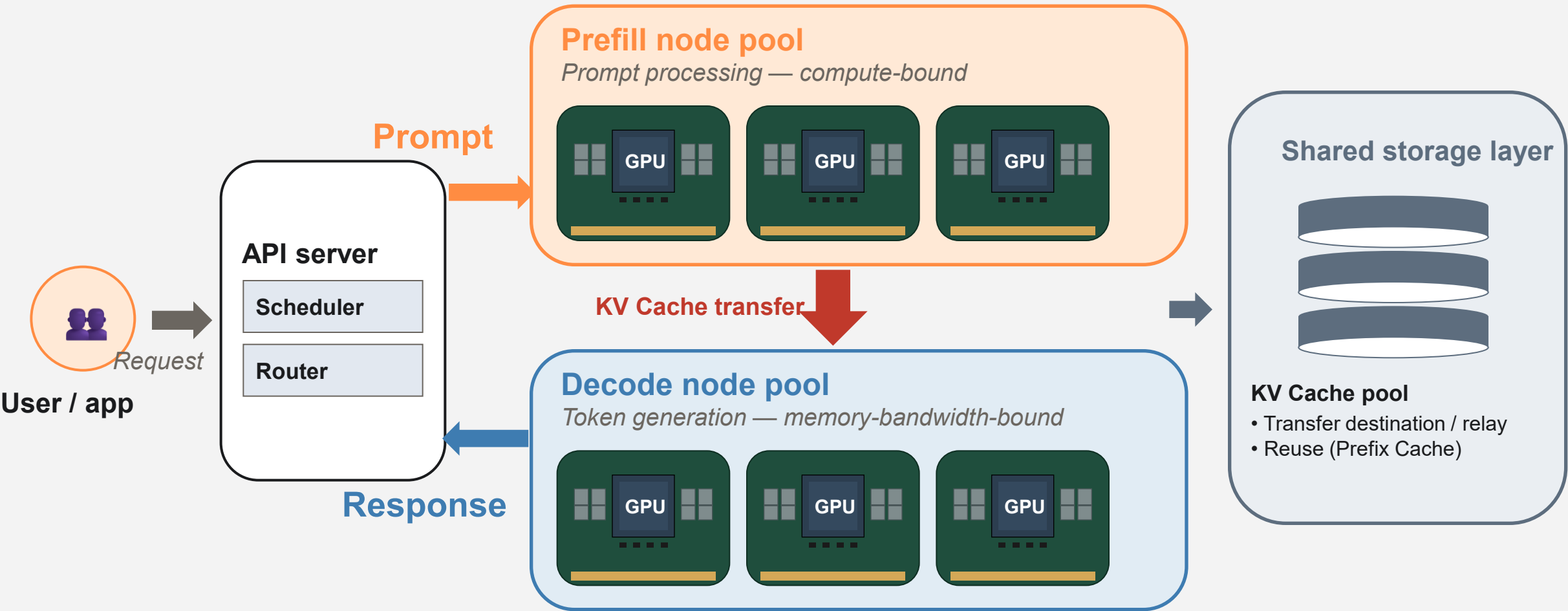
LLM Inference Workflow: Prefill and Decode



LLM Inference Workflow: Prefill and Decode (and KV Cache)

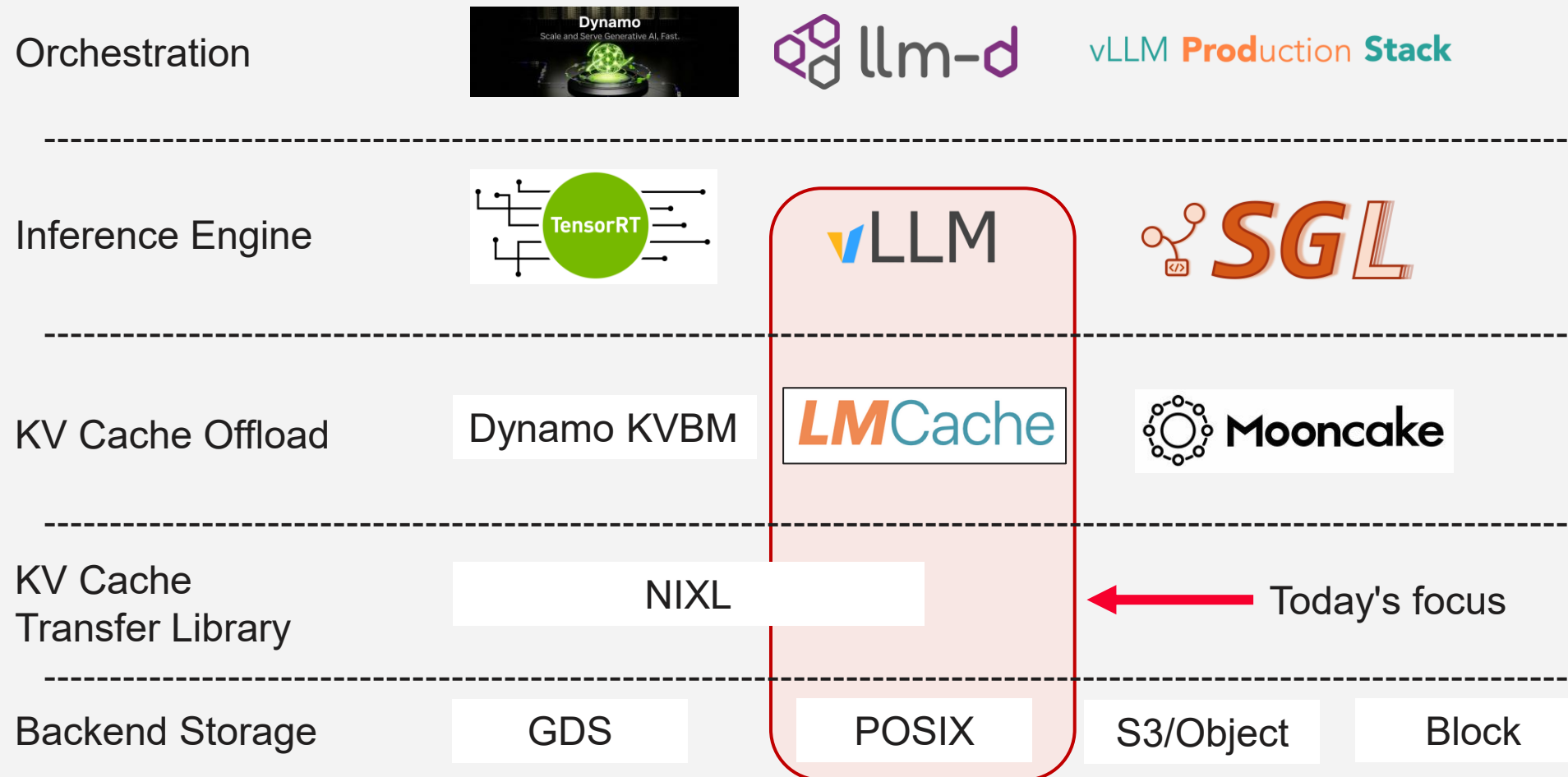


Disaggregating Prefill and Decode

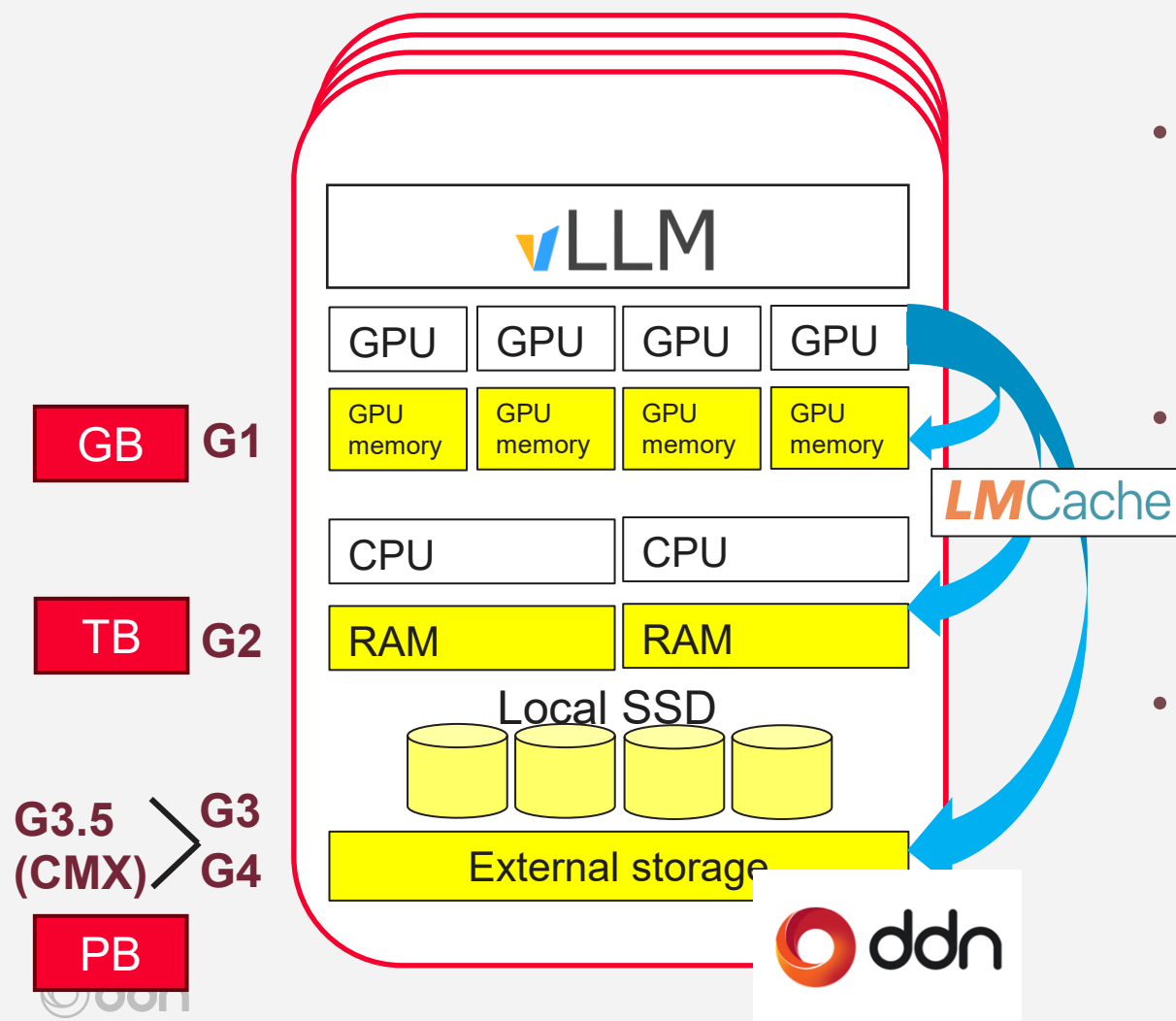


Separate Prefill (compute-bound) and Decode (memory-bandwidth-bound) into distinct pools to maximize GPU efficiency

AI Inference Stack



Storage Devices for KV Cache



- Any device that can store data is storage
 - GPU memory, host memory
 - Local disk, external storage
- KV Cache in GPU memory
 - Essential for efficient inference performance
 - Finite size on the order of GB
- Offload to other storage tiers
 - Transparent cache data management across memory and storage

KV Cache Impact - Model Size and Architecture

$$\text{KV/token} = 2 \text{ (K/V)} \times \text{num_kv_heads} \times \text{head_dim} \times \text{num_layers} \times \text{bytes_per_element}$$

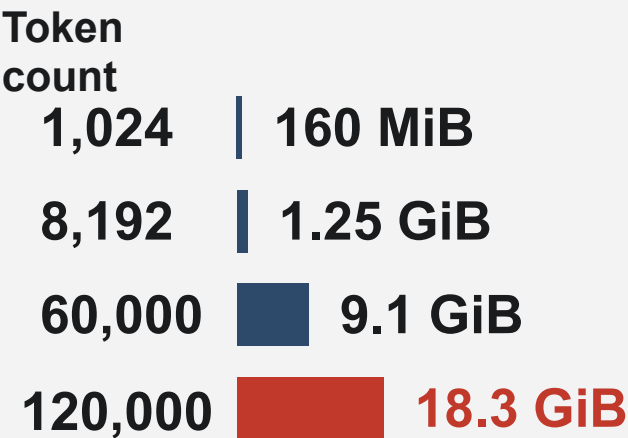
Model	Layers	KV heads	Head_dim	KV/token
GPT-3-175B (OLD)	96	96(MHA)	128	~4.7 MB
Qwen3-14B	40	8 (GQA)	128	160 KB
Qwen3-32B	64	8 (GQA)	128	256 KB
Llama-3.1-405B	126	8 (GQA)	128	516 KB
DeepSeek-V3 (671B)	61	MLA	-	~70 KB

At the same 128K context length, KV cache size varies by several to tens of times across models.

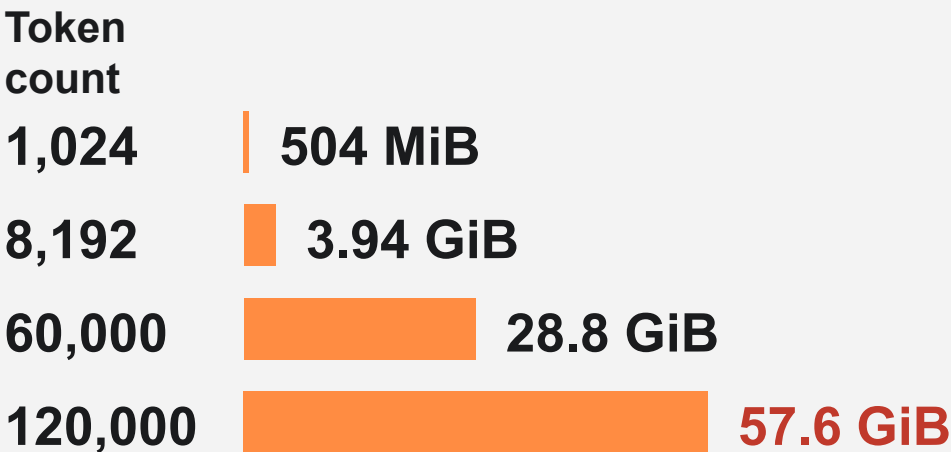
Models themselves are evolving to reduce KV cache footprint.

KV Cache Impact - Token Count

Model: Qwen3-14B



Model: Llama 3.1-405B



KV cache scales linearly with token count

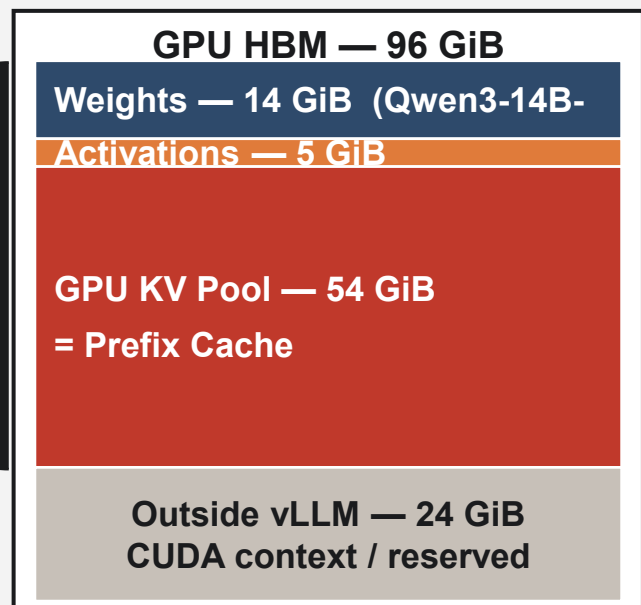
Inference Demo: KV Cache Impact in an Inference Environment

- **1 x NVIDIA GH200**
 - NVIDIA Superchip (Grace CPU/ARM64 + Hopper GPU)
 - 96GB HBM3 + 480GB DDR5 RAM
 - 1 x 400Gb Ethernet/RoCEv2
- **1 x DDN AI400X3 (100GB/sec Write, 130GB/sec Read)**
 - DDN EXAScaler
 - 24 x NVMe and 4 x 400Gb Ethernet/RoCEv2
- **Software stack**
 - vLLM (0.18.0)
 - LMCache (0.4.6) with NIXL-POSIX (1.1.0)
 - Model Qwen3-14B

Inference Demo: Memory Footprint

Memory allocation after vLLM startup

vLLM
allocation —
72 GiB
(0.75)



vLLM startup `--gpu-memory-utilization 0.75`

→ Reserves 72 GiB out of 96 GiB HBM

KV Pool (54 GiB) is used directly as Prefix Cache

-  Weights — model weights (14 GiB)
-  Activations / Graphs — working memory (5 GiB)
-  GPU KV Pool — also serves as Prefix Cache (54 GiB)
-  Outside vLLM — CUDA context / reserved (24 GiB)

Prefix Cache = the GPU KV Pool itself

When full, old KV entries are evicted and recomputation occurs

Inference Demo: Scenarios

- Application: ask-pref.py
 - A chat box that ingests public Japan travel guides (Wikipedia + Wikivoyage, CC-BY-SA 4.0) and answers questions
 - e.g. "recommend one local dish", "best season to visit", "one signature sight, one hidden gem, one local dish"
 - All 47 prefectures of Japan used as the sample fleet
 - Issue one query to each ingested prefecture and aggregate the output across all 47 into a single table
 - Limited to ~118K tokens per prefecture (model context + memory constraints)
 - KV cache size: ~18.9 GiB per prefecture (scales with token count) and total ~880 GiB with 47 prefectures
- Compare behavior across different KV cache configurations:
 - GPU RAM only (no KV Cache offload)
 - GPU RAM + CPU RAM (offload to CPU RAM)
 - GPU RAM + CPU RAM + EXAScaler (offload to EXAScaler)

Inference Demo: Without KV Offload (GPU RAM Only)

Ingesting 2 prefectures of data (Total KV Cache: ~40 GiB -> fits in GPU RAM)

```
$ ./ask-pref.py --prompts-dir /ai400x3-1/llm-demo/user1/pref-prompts --limit 2
```

[illegible]

[INGEST] mean TTFT/prefecture: 19.32s TTFT (per prefecture): 19.3s

[2 prefectures]> One must-see attraction and a famous local food.

Querying 2 prefectures in parallel (workers=2)...

```
Querying 2 prefectures: 100%|██████████| 2/2 [00:01<00:00, 1.93pref/s]
```

=== Fleet table (2 prefectures) ===

Prefecture	Must See Attraction	Famous Local Food
Aichi	Nagoya Castle	Kishimen
Akita	Senshu Park	Kiritanpo Nabe

```
[mean TTFT/pref: 0.33s | wall: 1.0s]
```

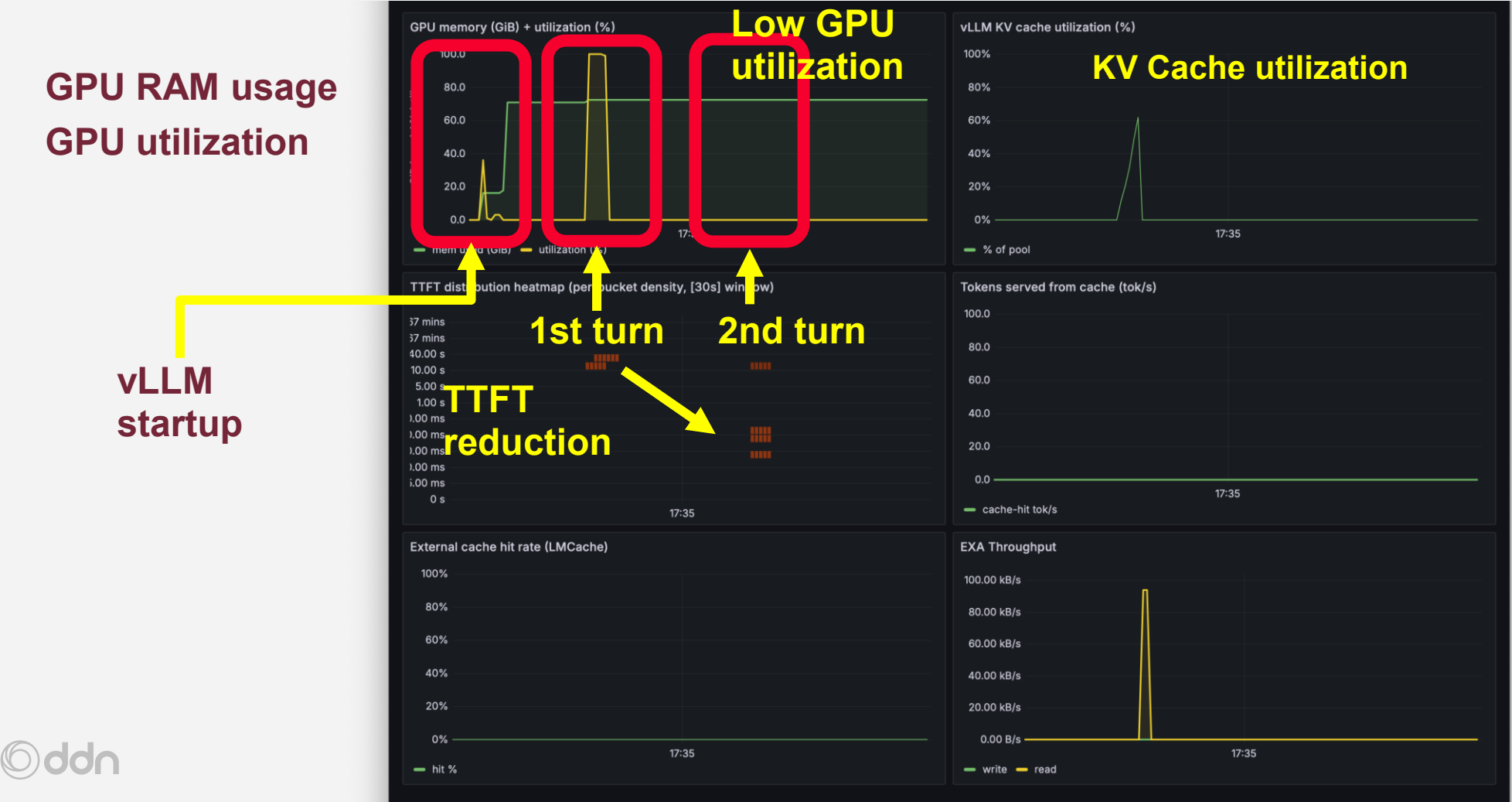
TTFT (per prefecture) cut from 19.3s to 0.33s

1st turn

1st-turn KV Cache is stored in GPU RAM

2nd turn

Inference Demo: Without KV Offload (GPU RAM Only)



 [mean]

```
$ ./ask-pref.py --prompts-dir /ai400x3-1/llm-demo/user1/pref-prompts --limit 5
```

1st turn

```
[5 prefectures]> Best season and ideal trip length
```

```
Querying 5 prefectures: 100%|████████████████████| 5/5 [01:02<00:00, 12.59s/pref]
```

2nd turn

Prefecture	Best Season	Ideal Trip Length
Aichi	autumn	5 days
Akita	spring and autumn	5-7 days
Aomori	summer	7_days
Chiba	spring	3_days
Ehime	spring and autumn	3 to 5 days

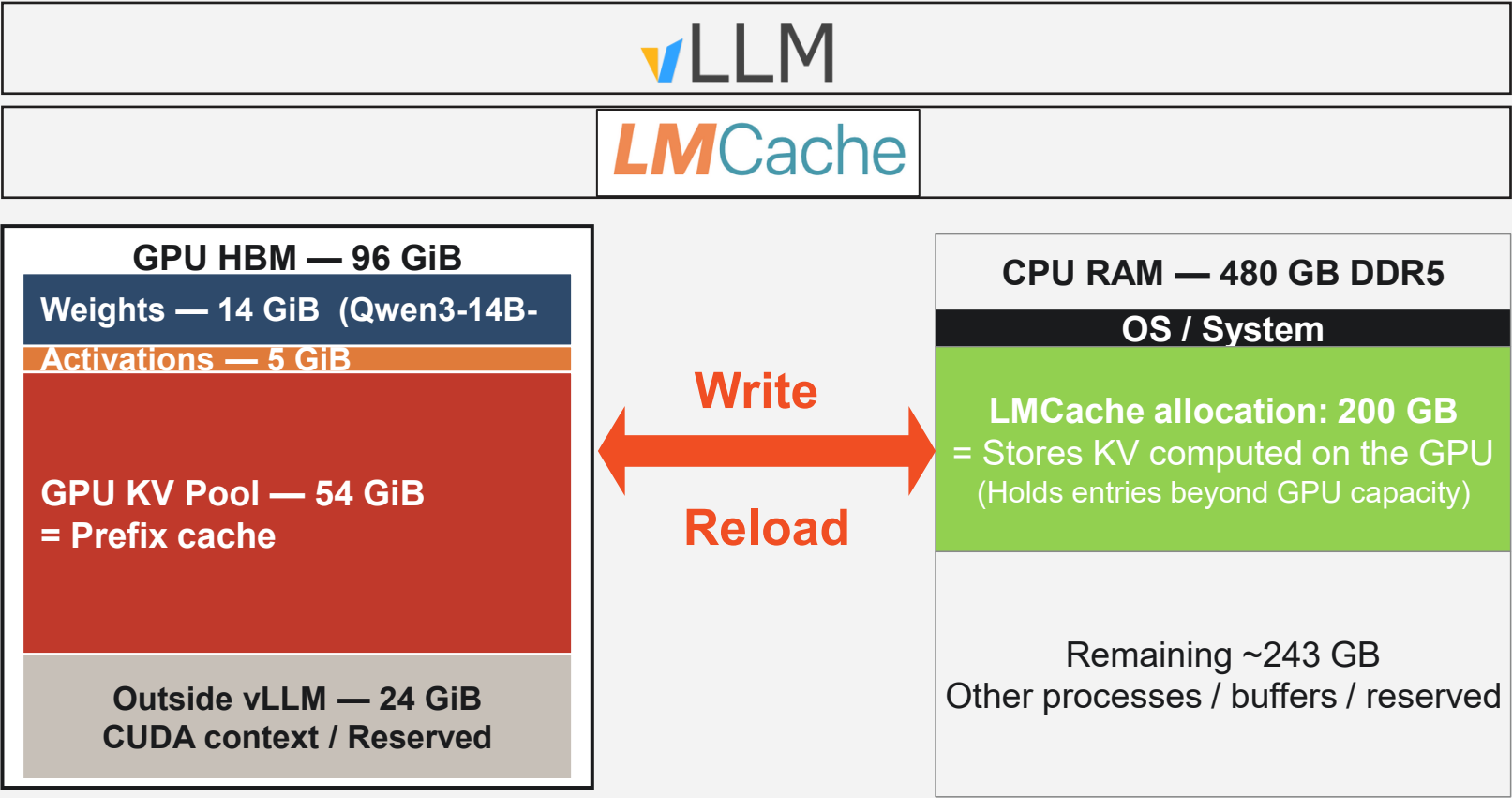
[mean TTFT/pref: 17.00s | wall: 62.9s] **TTFT per prefecture: 17s**

Inference Demo: When KV Cache Exceeds GPU Memory

Without enough GPU RAM, KV Cache cannot be reused, so the TTFT reduction benefit is lost



Inference Demo: Offloading KV Cache to CPU RAM

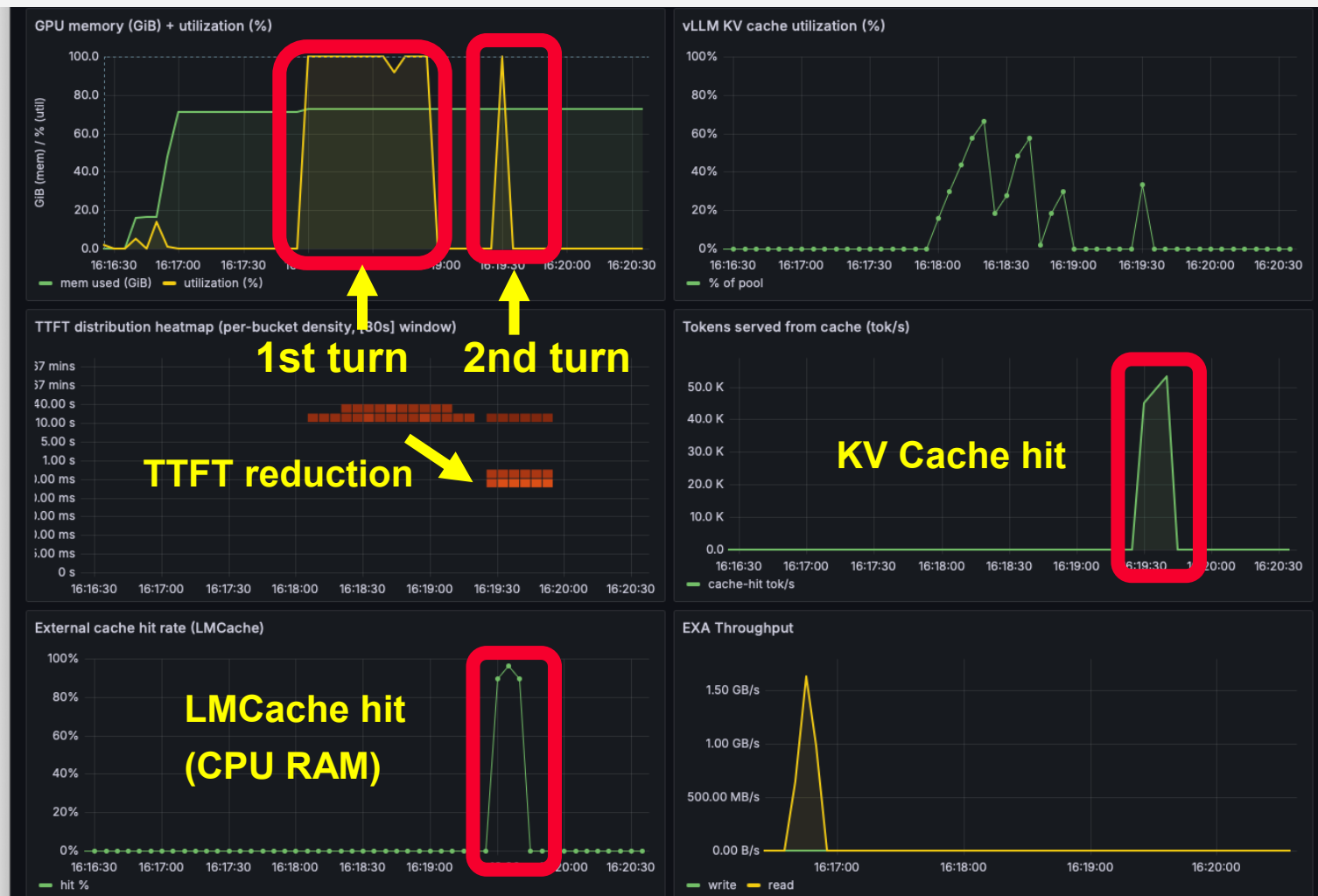


LMCache stores the KV Cache in CPU RAM and reloads it to the GPU on reuse

 [mean

**TTFT on reload
—from CPU RAM:
~16s**

Inference Demo: Offloading KV Cache to CPU RAM



2nd turn

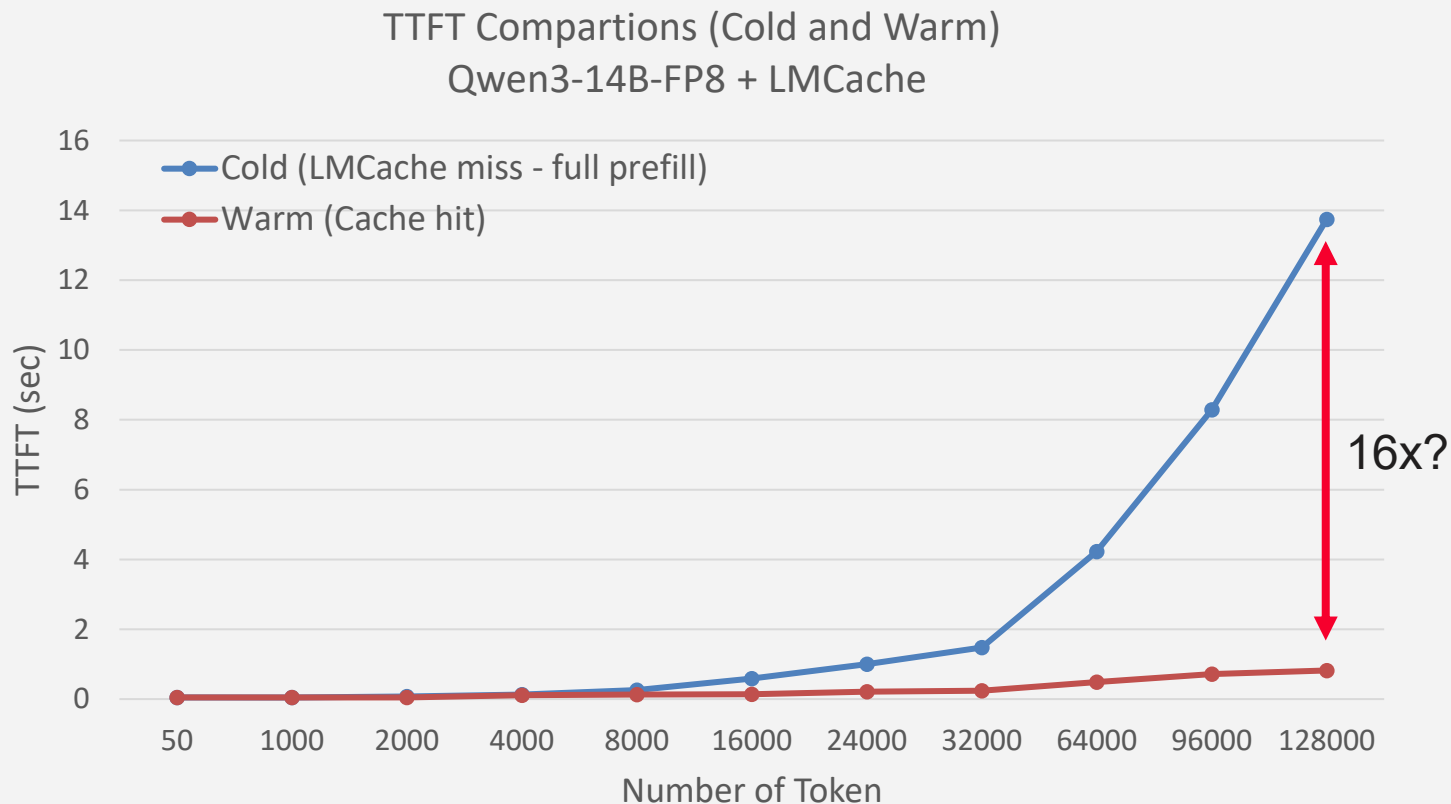
Inference Demo: When KV Cache Doesn't Fit in GPU or CPU Memory

When it no longer fits in CPU RAM either, a cache miss occurs and recomputation is required



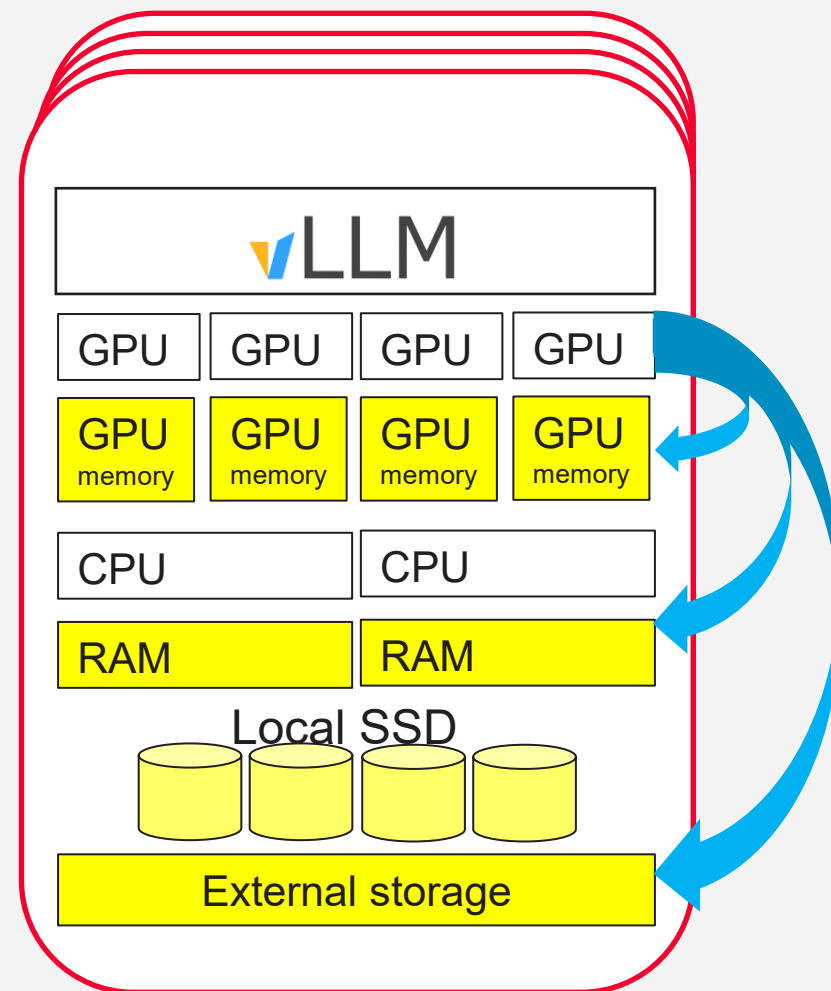
Why DDN?

Where Is the KV Cache Read From?



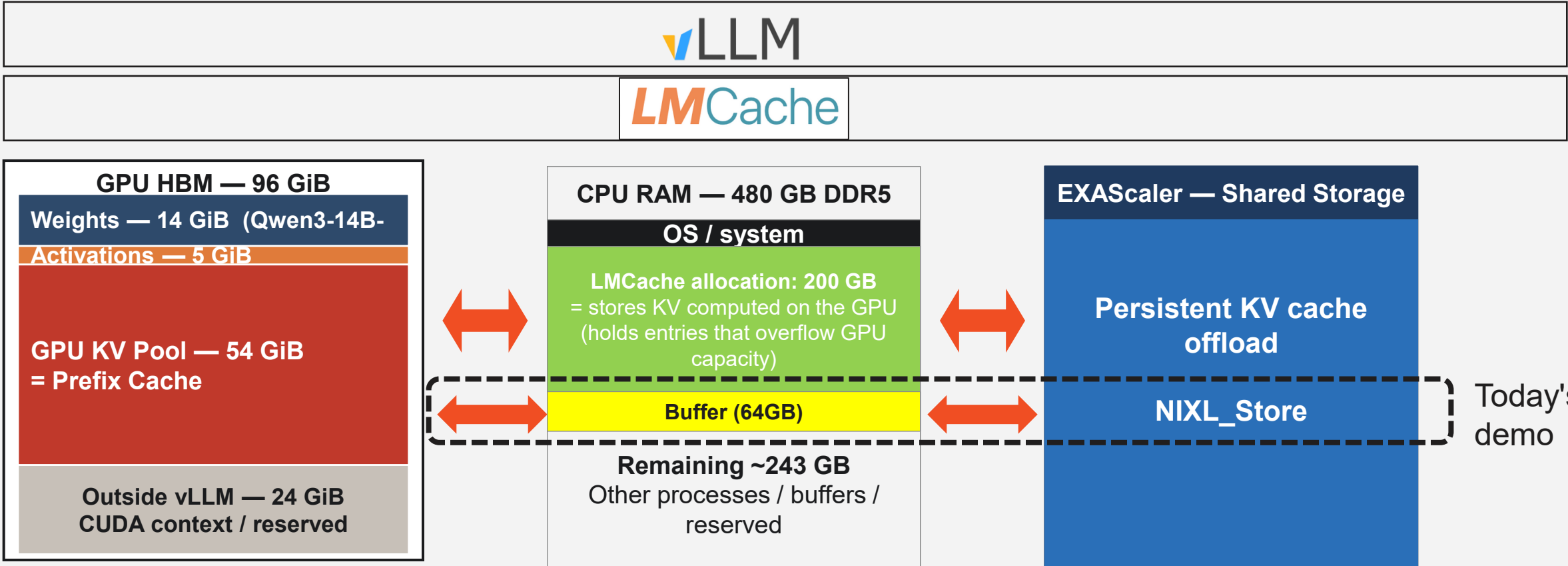
KV Cache reuse performance depends heavily on the storage tier.

Where Was the KV Cache Stored?



Offloading KV Cache to EXAScaler

LMCache writes computed KV Cache to EXAScaler, then reloads it to the GPU on reuse.



Inference Demo: KV Cache Offload to EXAScaler

Ingesting 20 prefectures of data (~400 GiB KV Cache > GPU+CPU RAM)

```
$ ./ask-pref.py --prompts-dir /ai400x3-1/llm-demo/user1/pref-prompts --limit 20
```

```
[INGEST]: 100%|████████████████████████████████████████| 20/20 [04:12<00:00, 12.64s/pref]=
```

```
[INGEST] mean TTFT/prefecture: 19.29s TTFT per prefecture: 19.2s
```

```
[20 prefectures]>For first-time visitors: one must-see spot and how many days needed
```

```
Querying 20 prefectures in parallel (workers=2)...
```

```
Querying 20 prefectures: 100%|████████████████████████████████████████| 20/20 [00:16<00:00, 1.24pref/s]
```

```
=== Fleet table (20 prefectures) ===
```

Prefecture	Top Sight	Days Needed
Aichi	Nagoya Castle	2 days
Akita	Lake Tazawa	2
Aomori	Sannai-Maruyama Site	2

```
[mean TTFT/pref: 1.03s | wall: 16.1s] TTFT per prefecture: 1.0s
```

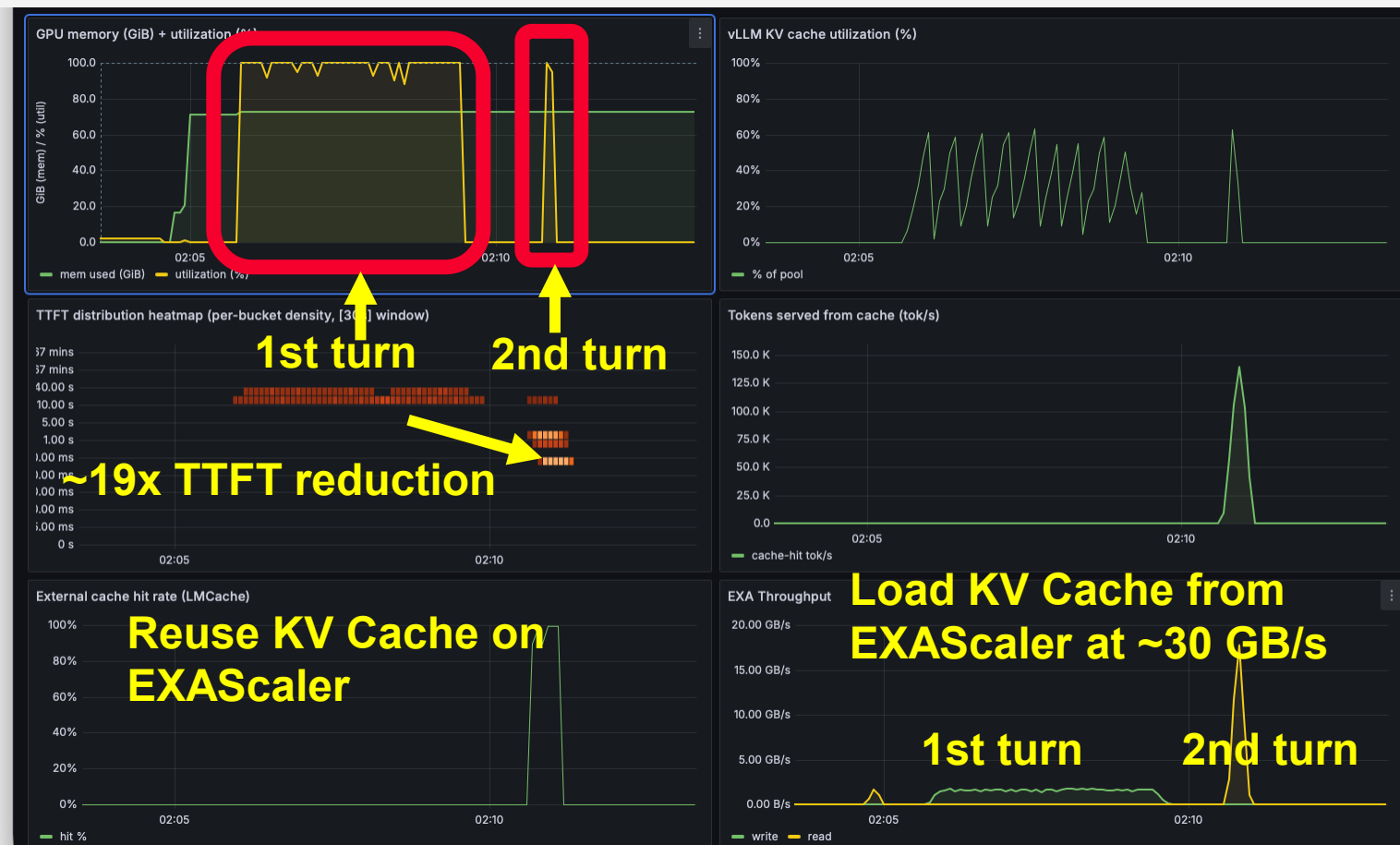
1st turn

2nd turn

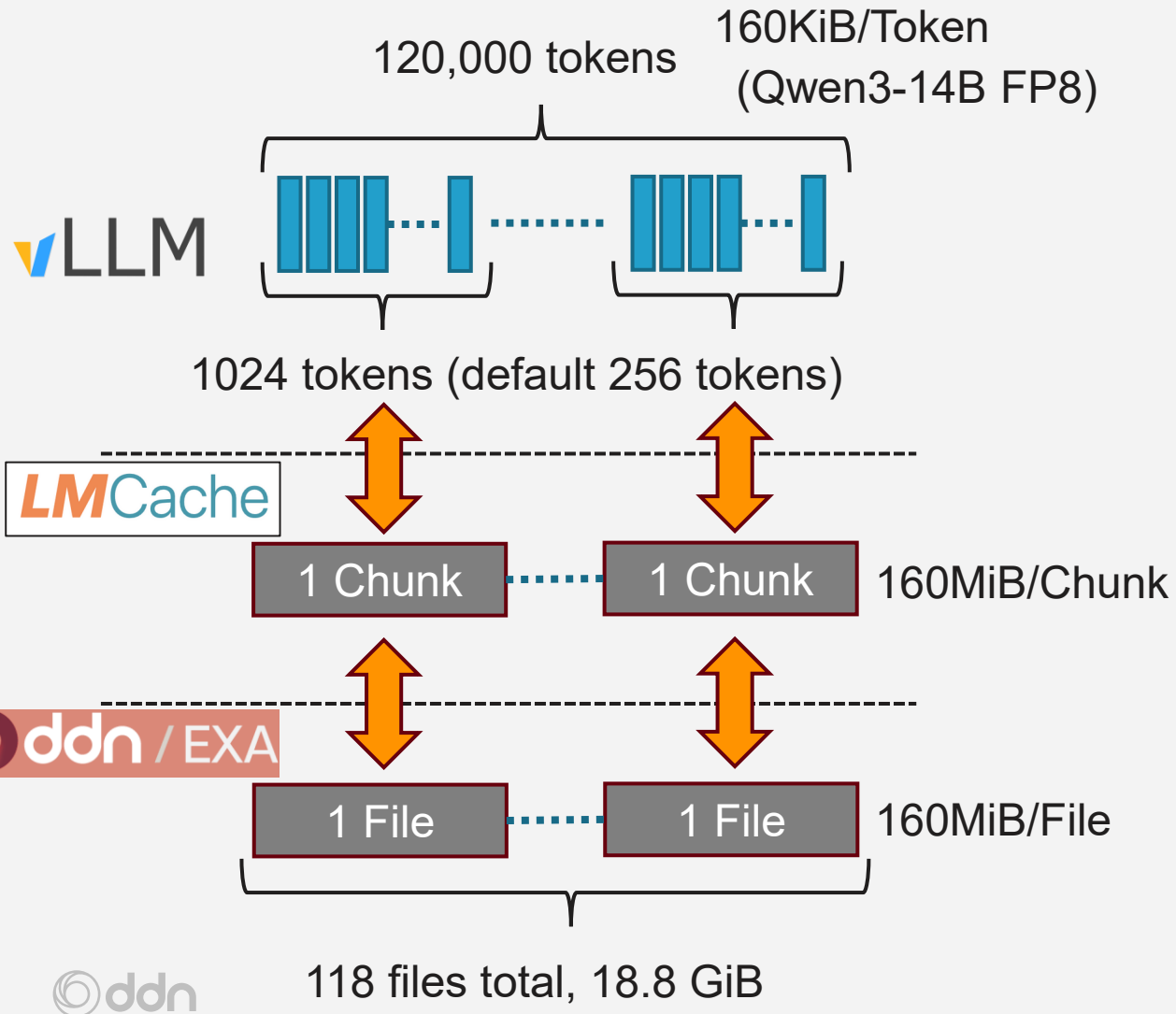
TTFT (per prefecture):
cut from 19.2s to 1.0s



Inference Demo: KV Cache Offload to EXAScaler



KV Cache Access Pattern Seen from EXAScaler

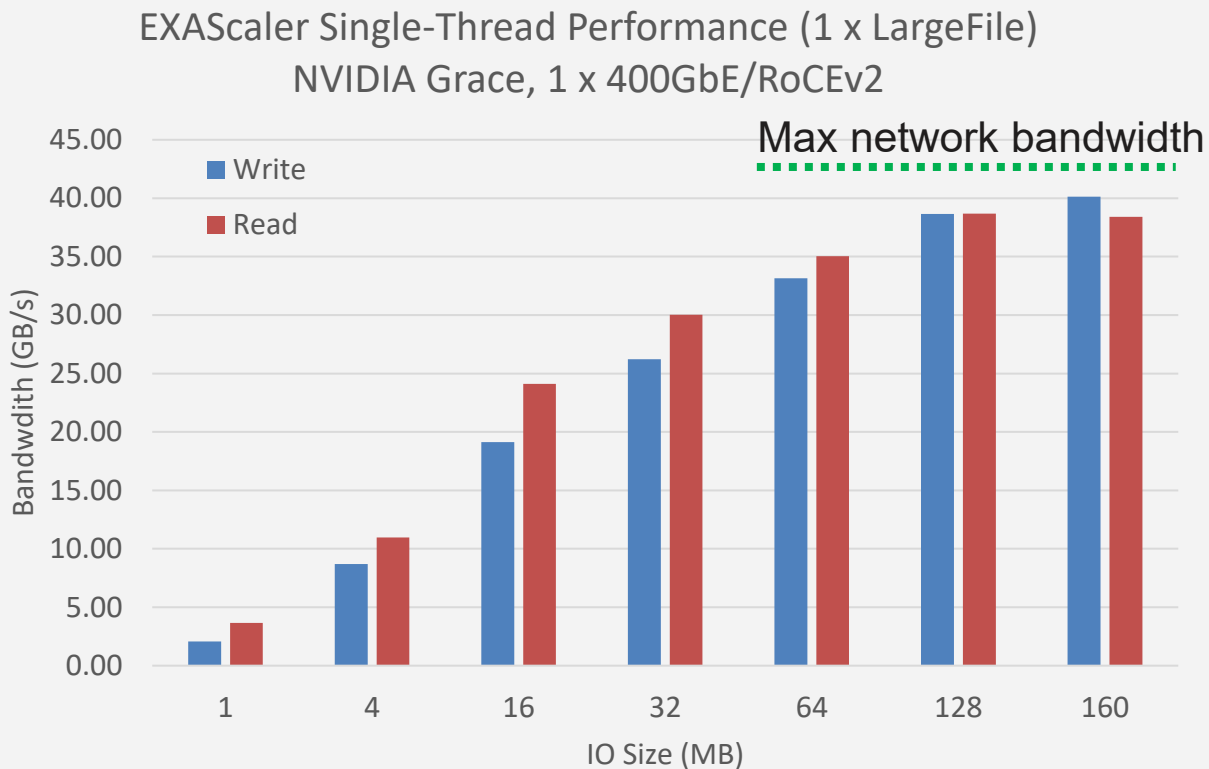


From the EXAScaler I/O trace

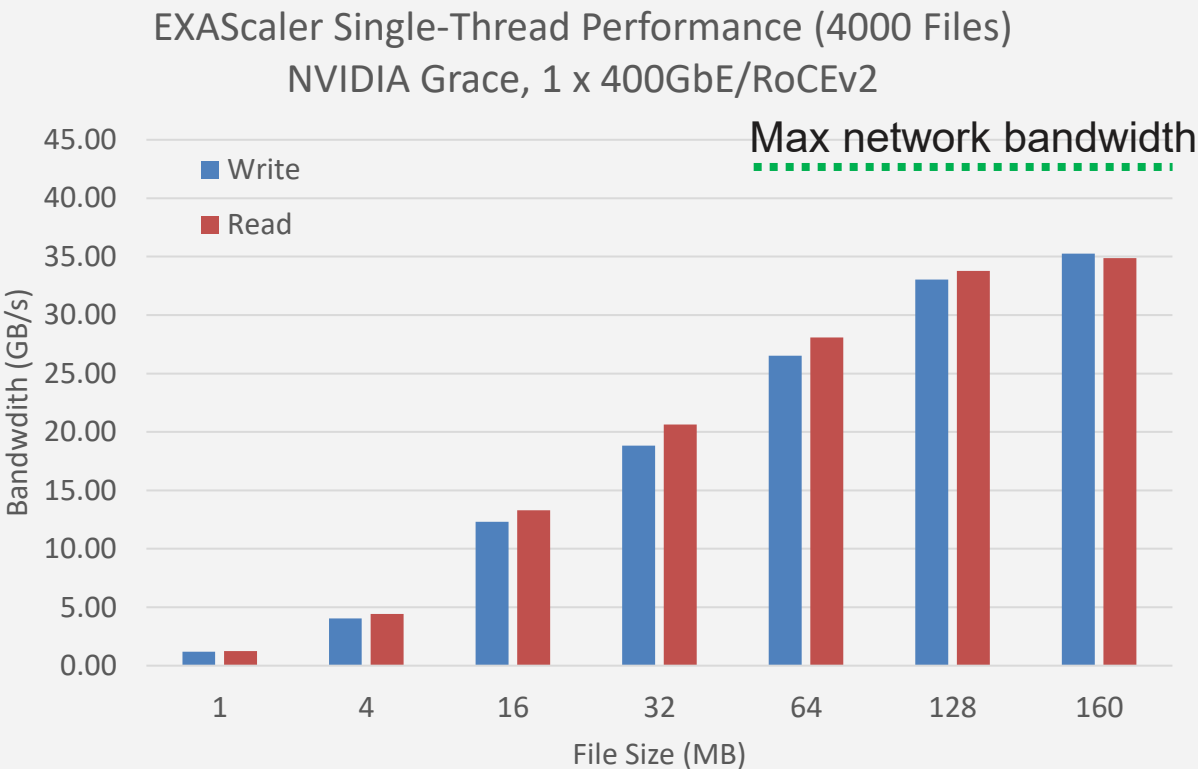
- 97% reads (1277/1317)
- 99.9% at 160 MiB I/O size
- Read latency: 4.56 ms (p50)
- Bandwidth per read: 34 GiB/s
- I/O runs sequentially (no parallel reads)

EXAScaler: Single Thread Performance (1 x 400Gbps Case)

Single process accessing 1 file



Single process accessing 4,000 files



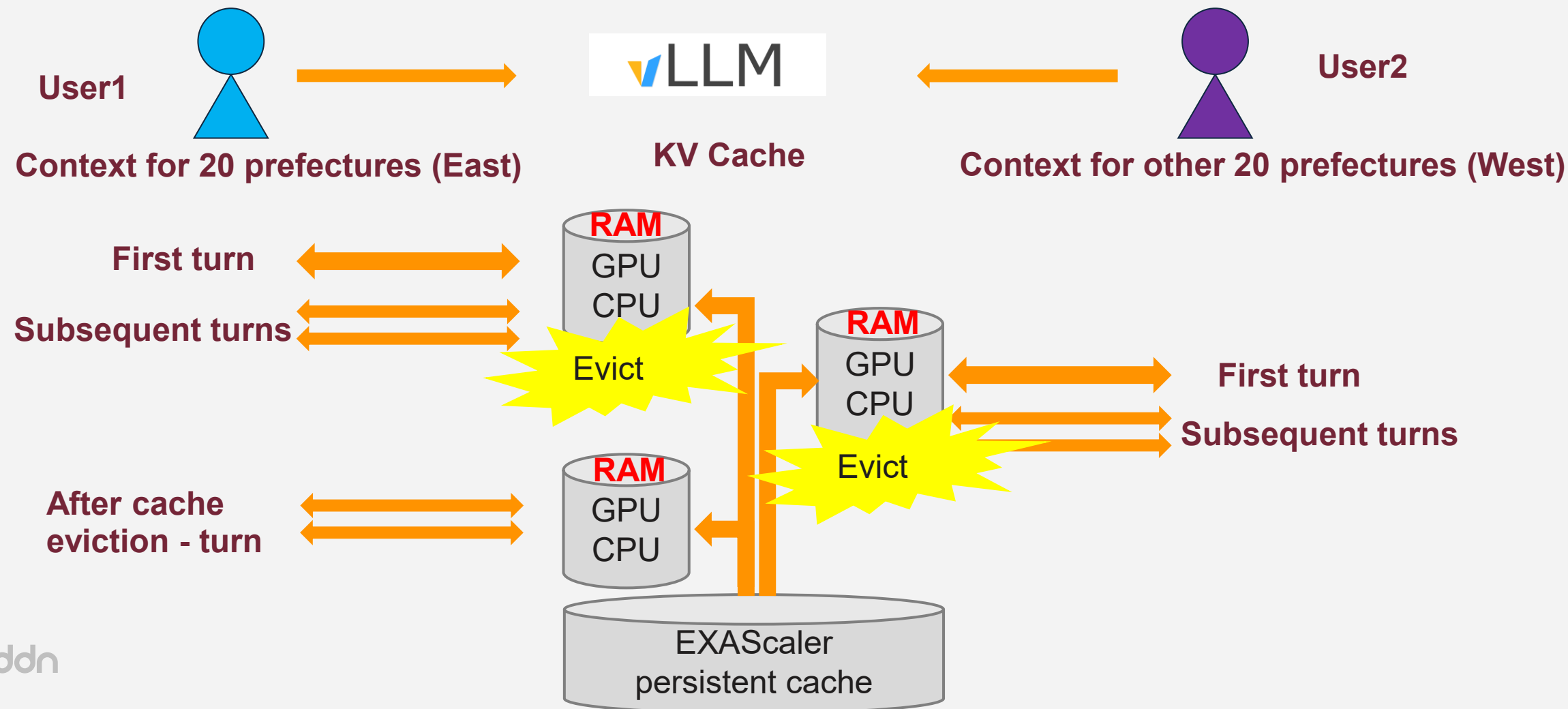
```
fio --name=psync1f_read_160m --ioengine=psync --iodepth=1 --rw=read --filename=/ai400x3-2/llm-demo/fio-test/testfile_160m --filesize=160m --size=160m --bs=160m --direct=1 --numjobs=1 --group_reporting --time_based --runtime=60 --fallocate=0 --scramble_buffers=0 --output-format=json
```

```
fio --name=psync_write_160m --ioengine=psync --iodepth=1 --rw=write --directory=/ai400x3-2/llm-demo/fio-test --nrfiles=4000 --filesize=160m --openfiles=128 --bs=160m --direct=1 --numjobs=1 --group_reporting --fallocate=0 --scramble_buffers=0 --file_service_type=random --output-format=json
```



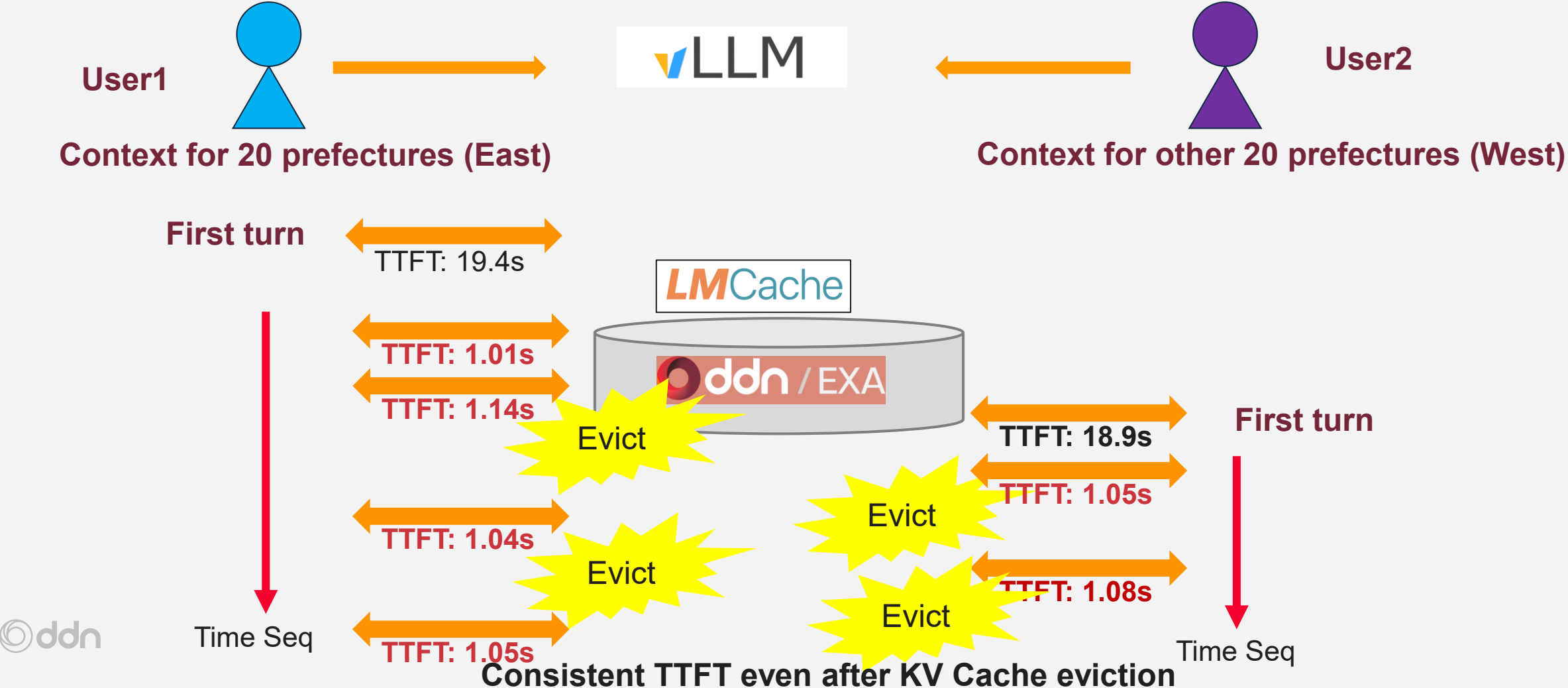
Inference Demo: EXAScaler KV Cache Offload

Multi-user, multi-turn scenario



Inference Demo: EXAScaler KV Cache Offload

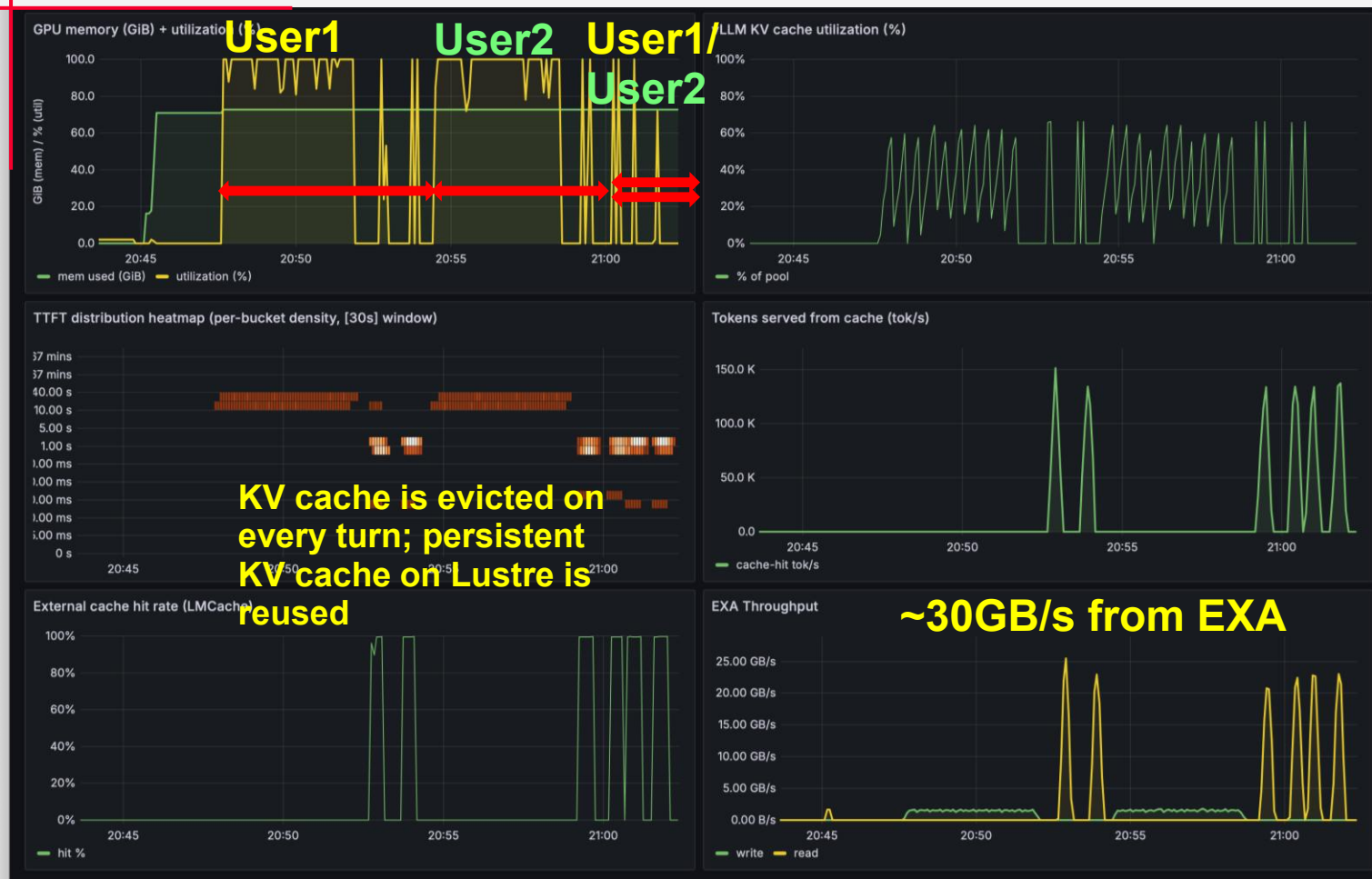
Multi-user, multi-turn scenario



Inference Demo: EXAScaler KV Cache Offload

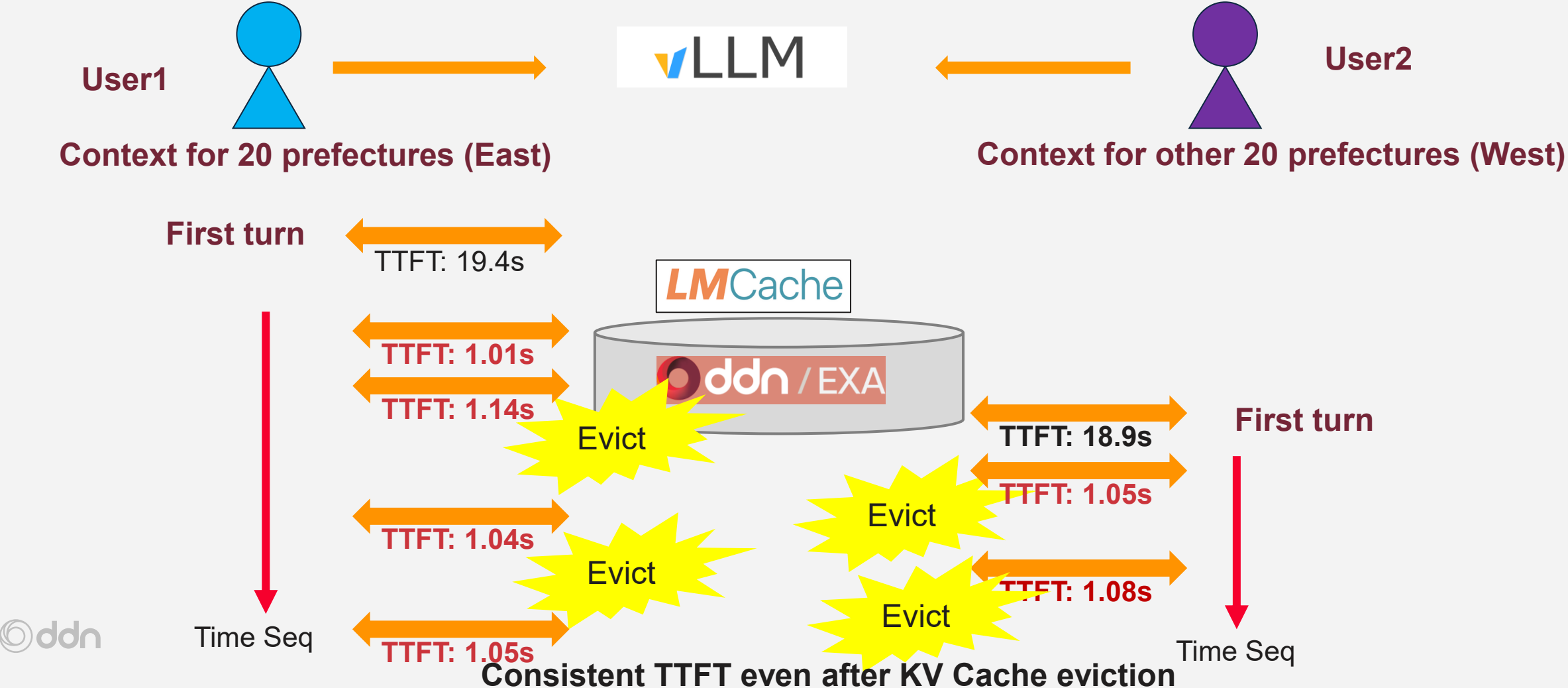
In practice, beyond I/O bandwidth:

- GPU utilization
- GPU memory bandwidth are also affected



DDN Ultrafast KV Cache Offloading helps for CAG

Multi-user, multi-turn scenario



Inference Performance

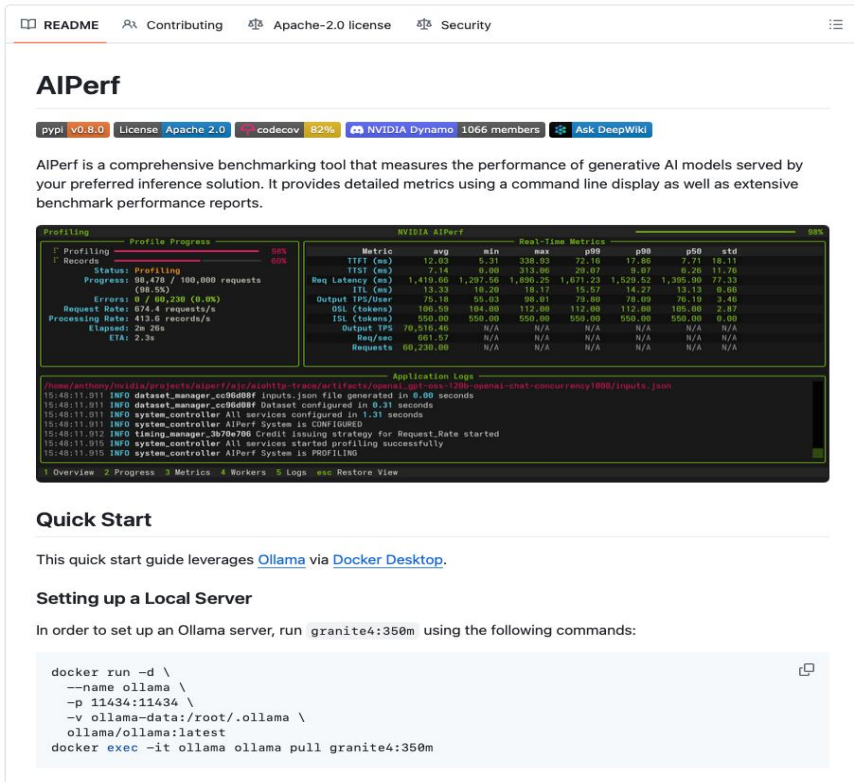
Interleaved Prefill + Decode Workload

Inference Demo: AIPerf - Workload-generating Inference Benchmark

NVIDIA AIPerf

NVIDIA's open-source benchmarking tool for generative AI inference (formerly GenAI-Perf)

- Production-like load generation
 - Flexible configuration of concurrency, ISL/OSL, and request rate
- Key metrics
 - TTFT, ITL, latency, Tokens/sec, Requests/sec
- Broad backend support
 - vLLM, TensorRT-LLM, NIM, Triton, OpenAI-compatible APIs
- Interleaved workloads
 - Prefill and Decode run continuously interleaved, reproducing production-like behavior without phase separation



The screenshot shows the NVIDIA AIPerf GitHub repository page. The repository is titled "AIPerf" and is licensed under Apache 2.0. It has 1066 members and is tagged with "py: v0.8.0". The description states that AIPerf is a comprehensive benchmarking tool for generative AI models, providing detailed metrics and benchmark performance reports.

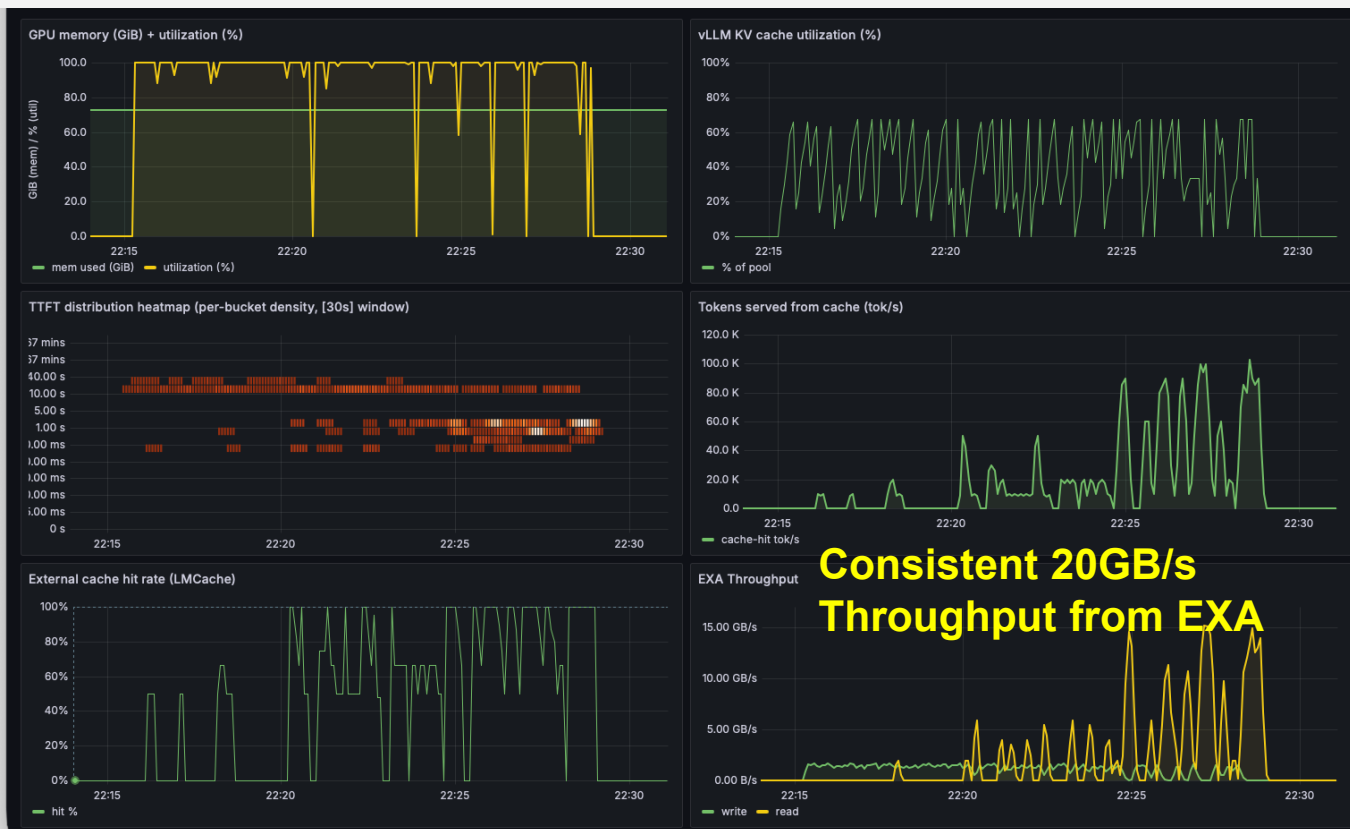
Below the description, there is a terminal window displaying the output of the AIPerf benchmark. The terminal shows the following metrics:

Metric	avg	min	max	p99	p50	std
TTFT (ms)	12.03	5.31	338.93	72.16	17.86	7.71
ITL (ms)	7.14	0.00	315.86	29.97	9.87	9.20
Req Latency (ms)	1,419.60	1,297.50	1,890.25	1,671.23	1,520.52	1,305.00
ITL (ms)	12.32	10.20	18.17	35.97	14.27	12.13
Output TPS/User	75.18	55.03	98.01	79.89	78.09	76.19
OSL (tokens)	100.50	104.00	112.00	112.00	100.00	2.87
ITL (tokens)	550.00	550.00	550.00	550.00	550.00	0.00
Output TPS	70,516.40	N/A	N/A	N/A	N/A	N/A
Req/sec	601.57	N/A	N/A	N/A	N/A	N/A
Requests	60,230.00	N/A	N/A	N/A	N/A	N/A

The terminal also shows application logs, including the generation of the inputs.json file and the configuration of the dataset manager.

<https://github.com/ai-dynamo/aiperf>

Inference Demo - AIPerf: EXA KV Cache Offload

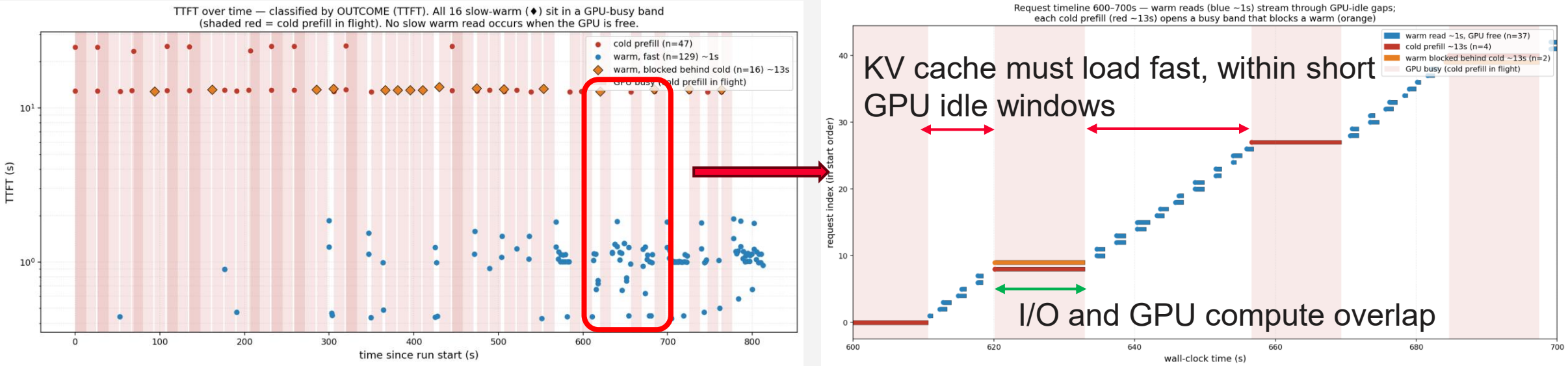


- **vLLM + EXA KV cache offload**
 - Hybrid KV cache:
200GB CPU RAM + EXA
- **Workload**
 - Model: Qwen3-14B-FP8
 - 48 documents (120K tokens each)
 - ~800 GB KV cache footprint
 - 192 total queries
 - Prefill and queries run continuously without separation

```
aiperf profile \  
  --model /ai400x3-2/llm-demo/models/Qwen3-14B-FP8 \  
  --url http://127.0.0.1:8000 \  
  --endpoint-type chat --streaming \  
  --tokenizer /ai400x3-2/llm-demo/models/Qwen3-14B-FP8 \  
  --synthetic-input-tokens-mean 120000 \  
  --prompt-input-tokens-stddev 0 \  
  --prompt-output-tokens-mean 100 \  
  --num-dataset-entries 48 \  
  --concurrency 2 \  
  --request-count 192 \  
  --dataset-sampling-strategy random \  
  --profile-export-prefix storage_pressure_48x120k_c2
```

Inference Demo: Inference evaluation with AIPerf

On a single GPU, prefill competes with queries



AIPerf c2 / Qwen3-14B-FP8 - 48 documents × 120K tokens, concurrency 2, 192 requests (random)

population	n	p50_s	p75_s	p90_s	mean_s
COLD (first touch, full prefill)	47	13.00	13.13	25.08	15.45
WARM (all cache hits)	145	1.09	1.26	12.83	2.38
↳ WARM, fast (GPU free / won race) ~1s	129	1.03	1.17	1.34	1.04
↳ WARM, blocked behind a cold prefill ~13s	16	13.16	13.27	13.35	13.17

Summary

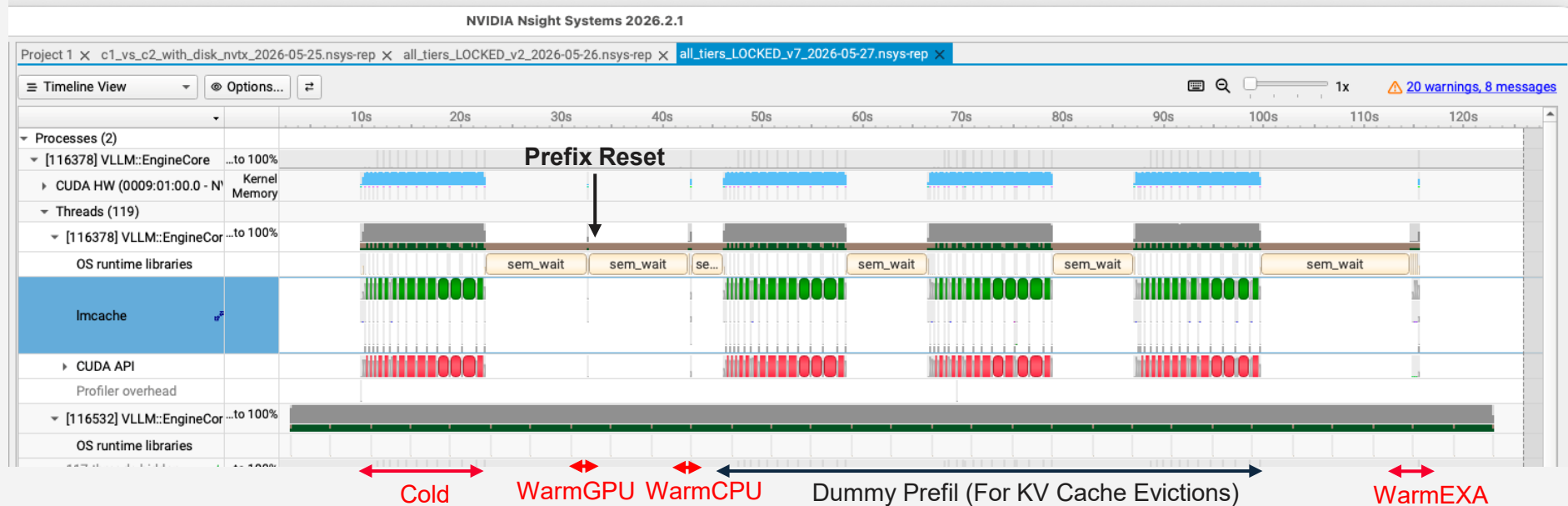
- Token throughput is the key to maximizing tokens/\$
 - Improve GPU utilization via Prefill-Decode disaggregation
 - Avoid recomputation by reusing a persistent KV cache
- Performance evaluation
 - End-to-end testing is essential
 - KV cache is affected by GPU and CPU configuration and state
 - Also depends on choice of GPU model, inference software stack, and transfer library
 - Beware of single-shot minimum TTFT numbers
 - Low TTFT can be reported even with low storage performance or light load, depending on how it is measured
 - Under high concurrent user load, other bottlenecks can mask the storage layer

LM Cache Performance Analysis

KV Cache Offload to EXA

KV Cache Offload : TTFT Comparisons

Latency Breakdown



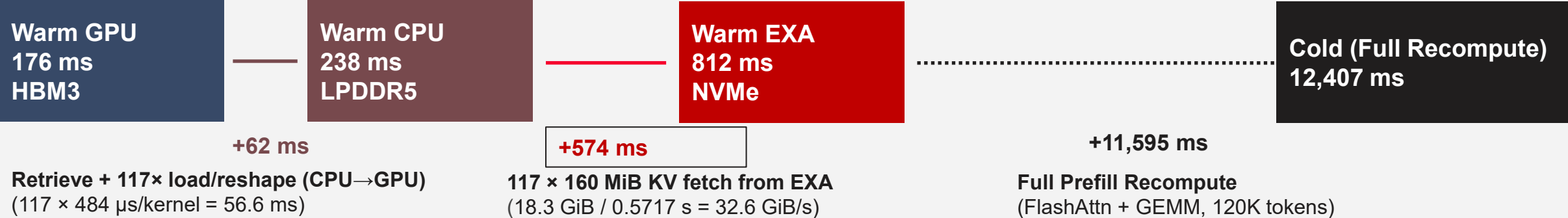
Workload Methodology

A single continuous **nsys** trace captures the same prompt across cache tiers:

- **Cold start:** fresh seed, no cached KV on disk or memory
- **Warm GPU:** vLLM prefix-cache hit
- **Warm CPU:** LMCache LPDDR hit after vLLM cache reset
- **Warm EXA:** KV restored from Lustre after CPU eviction using dummy prompts

KV Cache Offload : TTFT Comparisons

TTFT Decomposition by Cache Tier



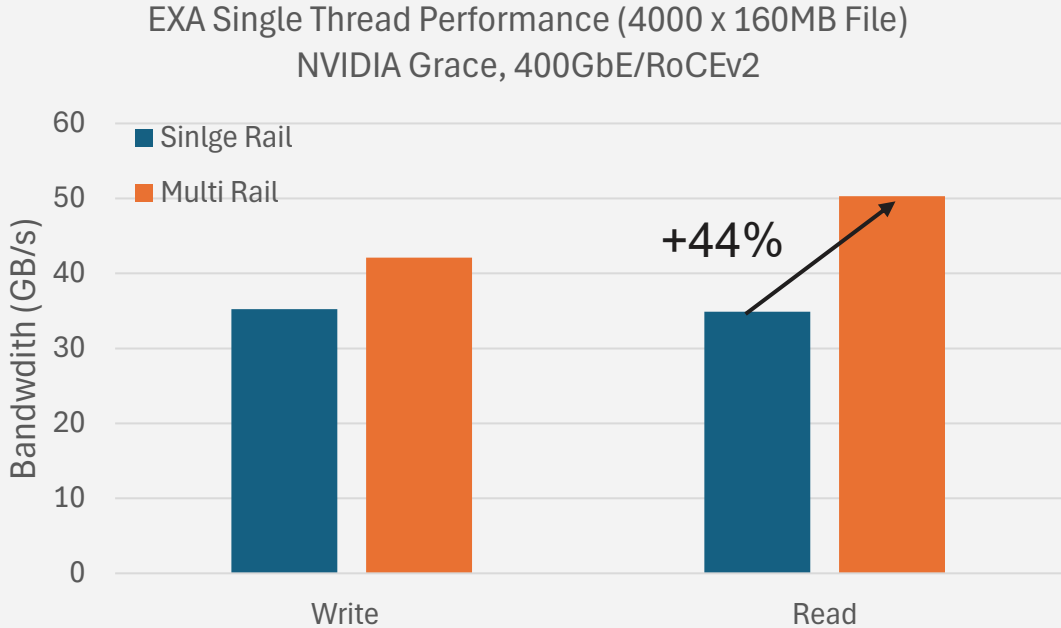
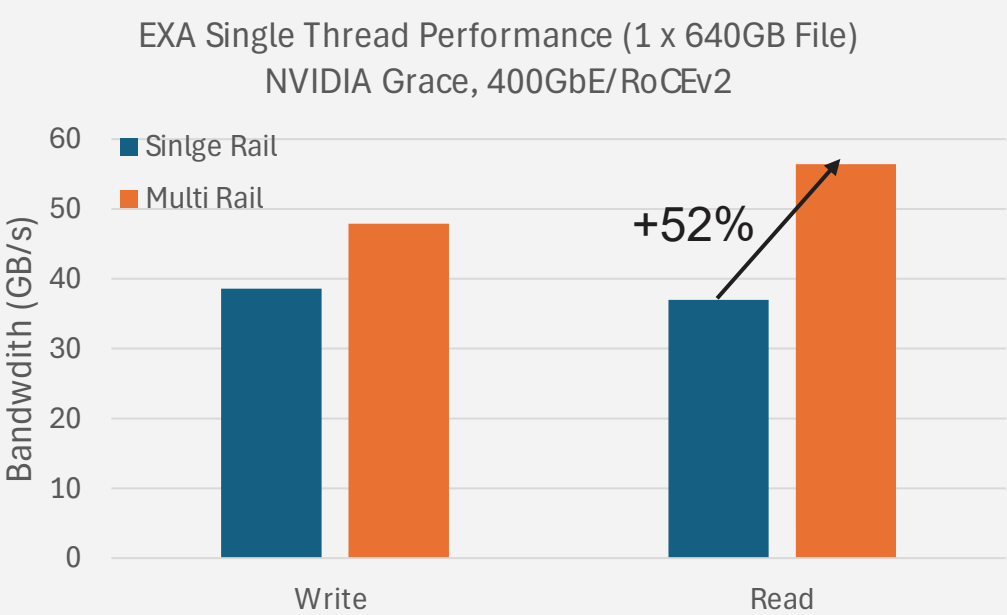
Nsight Systems Breakdown

Phase	WarmCPU	WarmEXA	Δ vs WarmCPU
Pre (anchor → first work)	21 ms	13 ms	-8 ms
KV Cache(19GB) loading from EXA	0 ms	571.7 ms	+571.7 ms
Retrieve + 117× load_and_reshape (to GPU)	58 ms	67 ms	+9 ms
First-token decode + return	159 ms	160 ms	+1 ms
e2e TTFT	238 ms	812 ms	+574 ms

EXA Restore: +574 ms vs. +11.6 s for full recompute

EXA KV Cache Offload: Multi-Rail Performance Gains

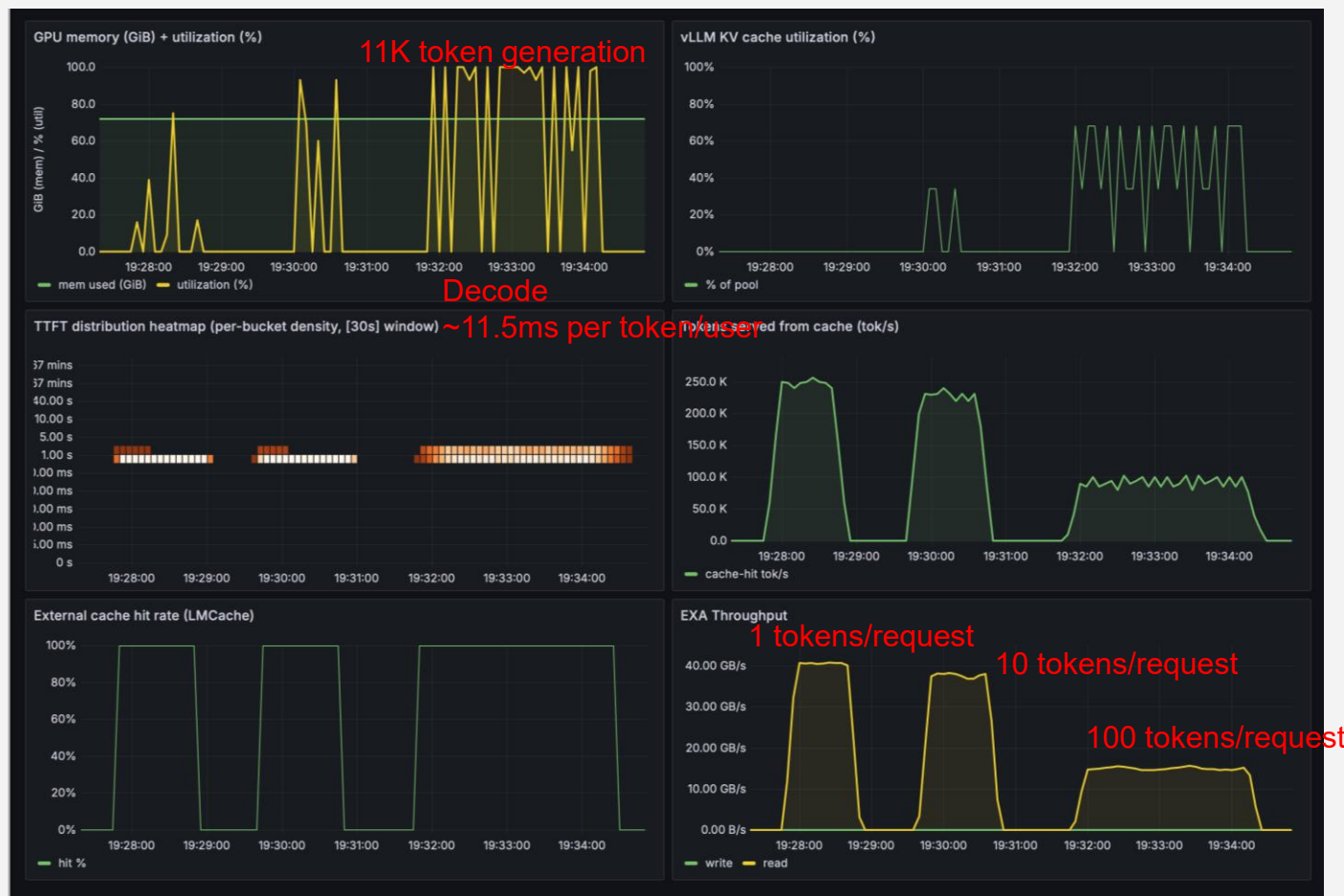
EXA Multi-Rail (2 × 400Gbps) on GH200 — Single-Thread Performance



Phase	WarmCPU	WarmEXA	WarmEXA (MR)
KV Cache(19GB) loading from EXA	0 ms	571.7 ms	399.6ms
e2e TTFT	238 ms	812 ms	655 ms

+44–52% bandwidth → 30% faster EXA restore

Why Output Length Matters in KV Cache Offload Evaluation



Workload

- 110 requests
- 120K input tokens each
- Model : Qwen3-14B FP8

If GPU token generation dominates, KV cache offload performance cannot be measured accurately.

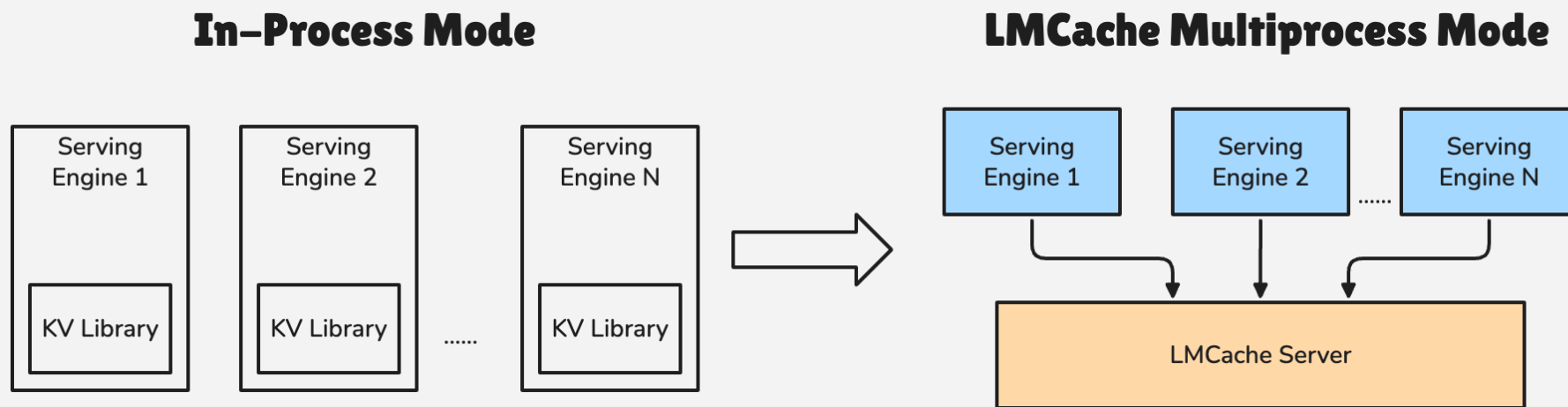
NIXL Static vs Dynamic Backend (Non-MP)

Non-MP (V1): Static vs Dynamic Backend

	NixlStaticStorageBackend	NixlDynamicStorageBackend
Storage Layout	Pre-allocated Object Pool	One File per KV Entry
Lookup	In-Memory Only	Filesystem Metadata
Capacity	Fixed (pool_size × chunk_size)	Disk Capacity
Eviction	Automatic (slot recycling)	None
Restart Recovery	No	Yes
Async Read	Not Supported today (Added by PR #3473)	Supported

Dynamic backend enables persistent KV cache, restart recovery, and storage capacity beyond a fixed pool.

In-Process vs. Multiprocess Mode



LMCache Deployment Models

- **In-Process Mode:** Each serving engine embeds its own LMCache library and manages KV cache independently.
- **Multiprocess Mode:** A dedicated LMCache server is shared by multiple serving engines through NIXL IPC.
- **Benefit:** Shared CPU/storage cache reduces memory footprint and improves cache reuse across inference engines.

NIXL Implementations in LMCache

Implementation Matrix

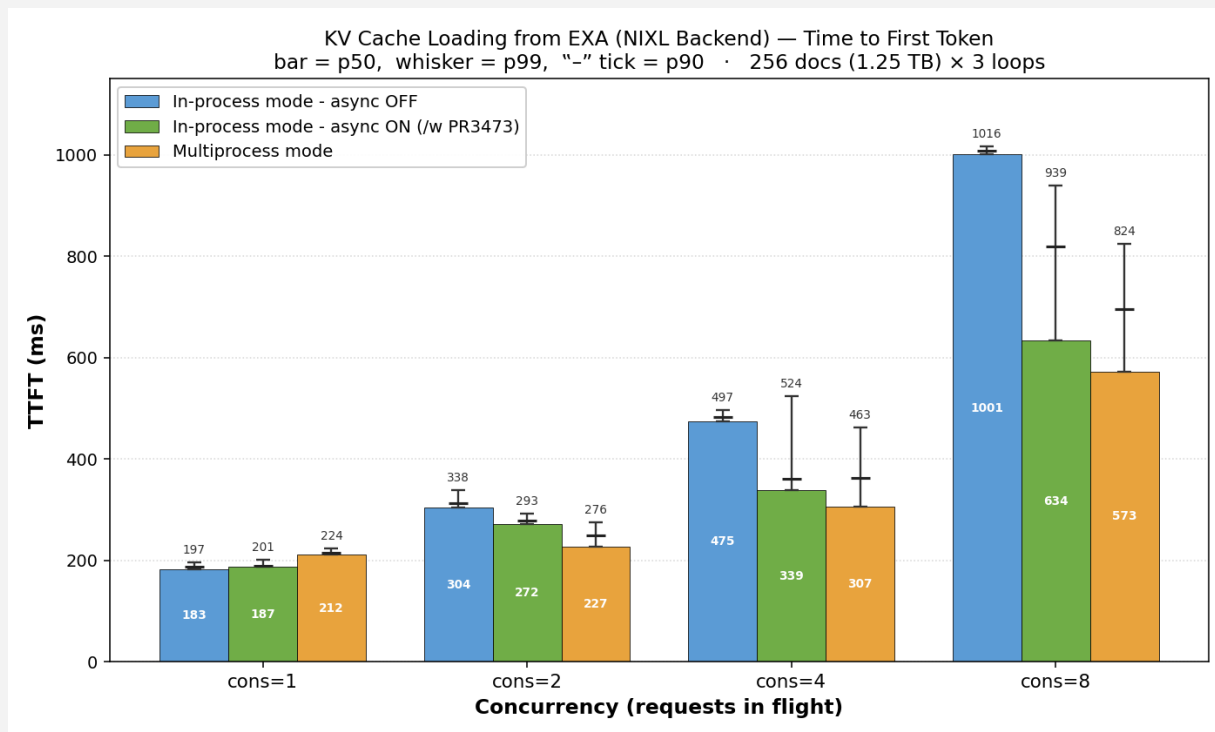
	In-process mode (V1)	MP(Multiprocess) mode (L2 Adapter)
Static (Fixed pool and volatile metadata)	NixlStaticStorageBackend	NixlStoreL2Adapter (Type nixl_store)
Dynamic (Persistent metadata and recovery)	NixlDynamicStorageBackend (NIXL_POOL=0)	DynamicNixlStoreL2Adapter (type nixl_store_dynamic)

Key Points

- Both In-Process and MP modes provide Static and Dynamic NIXL implementations
- Static mode uses a fixed pool with volatile metadata
- Dynamic mode maintains persistent metadata and supports restart recovery
- Both modes can use the same storage systems (e.g., EXAScaler and Infinia) via NIXL

TTFT with EXAScaler KV Cache Offloading (In-Process and MP modes)

EXAScaler KV cache offloading enables low TTFT



Both In-Process and MP modes achieve similarly low TTFT

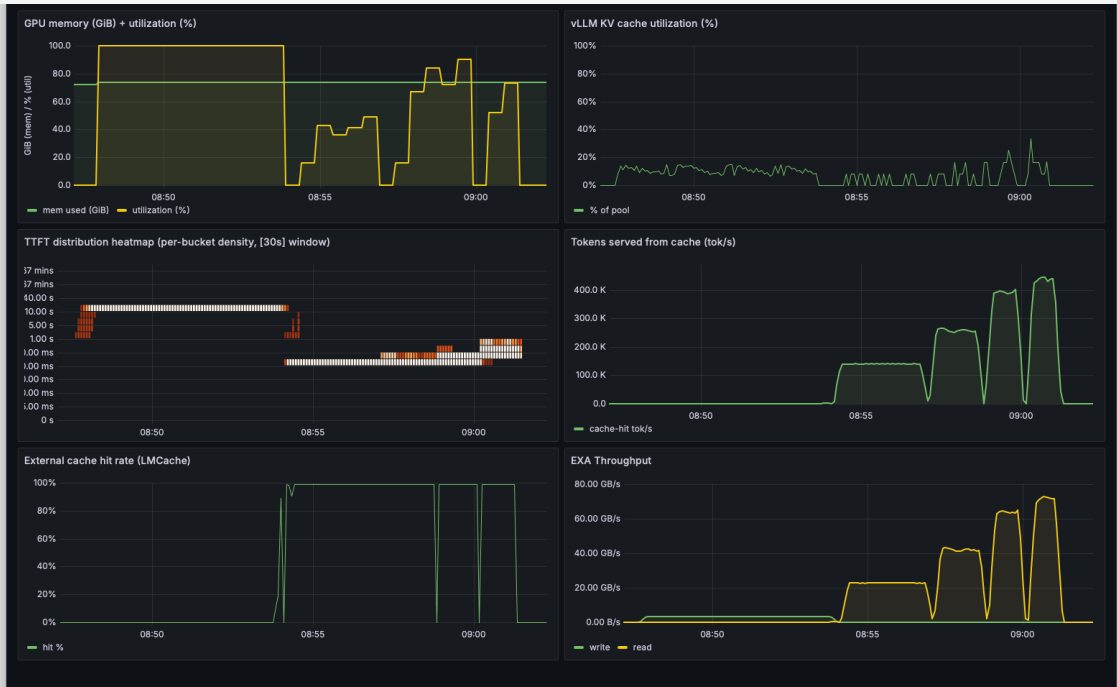
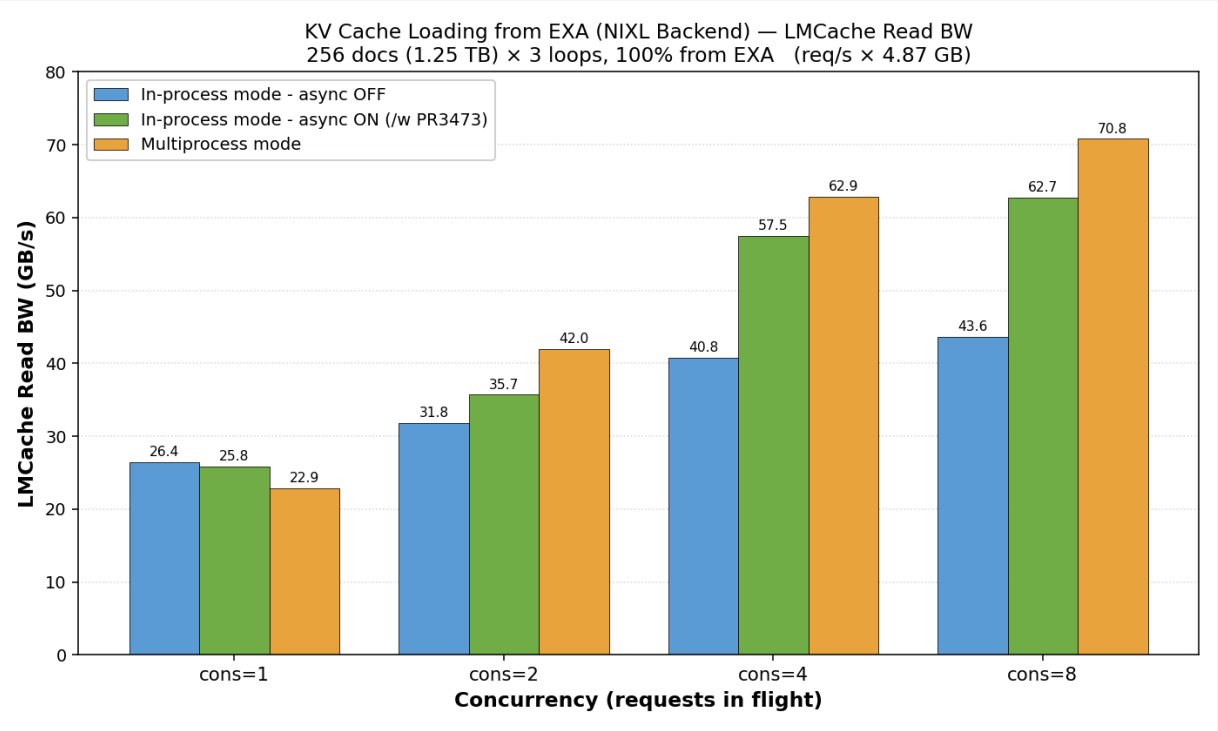
Configuration and workload

- Hardware
 - AI400X3M with EXA 6.3.8
 - GH200 with 2 x 400Gbps
- Software Stack
 - vLLM 0.18.0
 - LMCache 0.4.7
 - Model : Qwen3-14B-FP8
 - aiperf (30K input tokens, 1 output token to see KV cache loading speed)

```
aiperf profile --model Qwen3-14B-FP8-yarn --tokenizer Qwen3-14B-FP8-yarn ¥  
--endpoint-type chat --url http://127.0.0.1:8000 --streaming ¥  
--concurrency C --num-dataset-entries 64 --warmup-request-count 1 --request-count $((C*8)) ¥  
--synthetic-input-tokens-mean 30000 --synthetic-input-tokens-stddev 0 ¥  
--output-tokens-mean 1 --output-tokens-stddev 0 ¥  
--extra-inputs 'ignore_eos:true' --dataset-sampling-strategy sequential --random-seed 10008
```


KV Cache Loading Bandwidth with EXAScaler (In-Process and MP Modes)

EXAScaler KV cache loading can saturate the available network bandwidth



Up to 70.8 GB/s KV Cache Loading to a Single GPU

Performance Evaluation: Old vs. New Software Stack

- **Old Stack**

- vLLM-0.18.0
 - LMCache-0.4.7
 - PyTorch-2.10.0+cu129
 - CUDA-12.9
 - NIXL-1.2.0
- >
- New Stack**

- vLLM-0.24.0
 - LMCache-0.5.2-dev
(commit: 89f317a9)
 - PyTorch-2.11.0+cu130
 - CUDA-13.0
 - NIXL-1.3.0

Key Results (App BW @ Concurrency = 8, 100% Cache Hit)

	Baseline LMCache 0.4.7 (vLLM 0.18)	Patched LMCache 0.4.7 + #PR3696	New Stack LMCache 0.5.2-dev (vLLM 0.24) #PR3696 merged
Static Backend (nixl_store)	71.2 GB/s	71.2 GB/s	71.8 GB/s
Dynamic Backend (nixl_store_dynamic)	43.7 GB/s	69.6 GB/s	72.2 GB/s

- **Test Case (Hero L2 Performance)**
 - Qwne3-14B-FP8
 - 256 unique, 30K tokens each
 - Total KV cache size: 1.25 TB (256 × 4.87 GB)
 - 256 requests, one per document
 - Output length: 1 token
 - Concurrency levels: 1, 2, 4, and 8
 - Platform : AI400X3-M + EXA 6.3.8

Evaluation: Batched NIXL Request



ddn.com